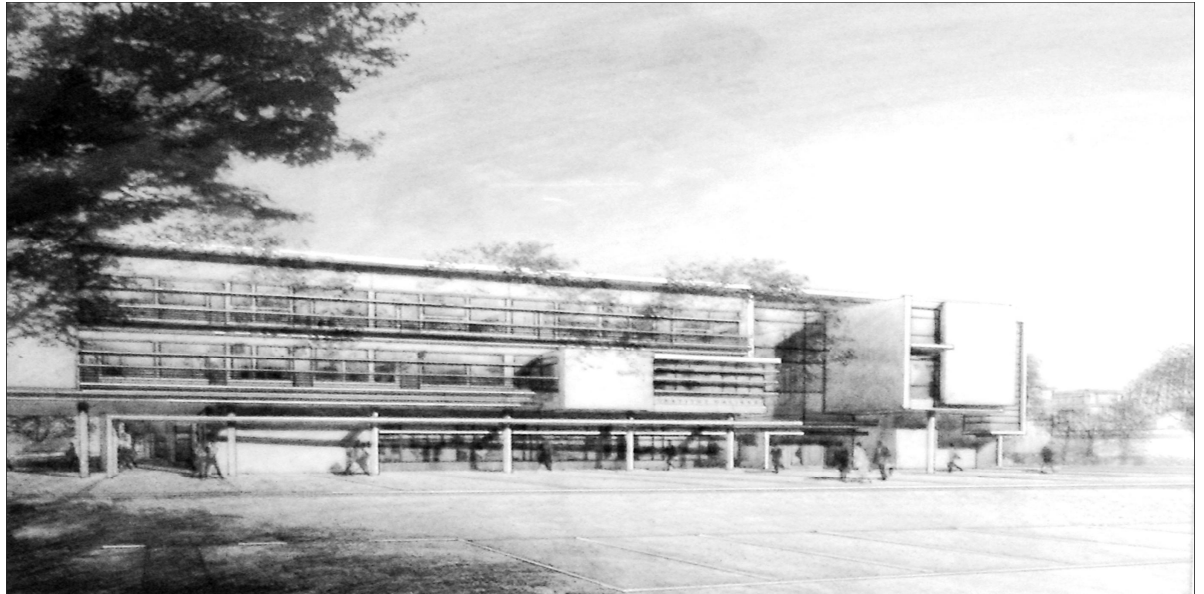




## Université Paris 13

### Ingénieurs Instrumentation 1<sup>ère</sup> année

### PROJETS PIC

### TEXTES DES PROJETS

#### DIFFERENTS PROJETS

|                             |    |
|-----------------------------|----|
| MOTEUR PAS A PAS.....       | 04 |
| TELEMETRE A ULTRA SONS..... | 21 |
| SERVOMOTEUR.....            | 34 |
| CLAVIER MATRICIEL.....      | 51 |
| GPS.....                    | 63 |
| MODULATION MLI.....         | 78 |
| GRADATEUR DE LUMIERE.....   | 93 |

**THIEBAUT Bernard**

# INTRODUCTION ET CONSEILS GENERAUX :

## A LIRE AVANT TOUTE CHOSE

Le projet permet de concrétiser et en quelque sorte de valider la série de Cours/TP sur le microcontrôleur PIC et ceci sur un exemple plus concret, plus complet et peut-être plus intéressant car plus proche des applications réelles.

Il consiste en la mise en œuvre d'un périphérique via une petite interface.

Cependant, dans chaque projet, il vous sera demandé, dans un deuxième temps, d'utiliser le LCD alphanumérique présent sur les cartes PICDEM 2 PLUS.

Par exemple, pour afficher :

- la vitesse du moteur pas à pas,
- la distance mesurée avec le télémètre ultra-sons,
- les informations fournies par le récepteur GPS,
- les caractères frappés sur le clavier matriciel,
- les modes de fonctionnement du servomoteur...

Il faut donc **impérativement** avoir terminé la partie prise en main du LCD du **TP3** pour pouvoir utiliser les fonctions de base permettant d'afficher un caractère ou une chaîne de caractères, pour positionner le curseur d'affichage n'importe où sur le LCD, pour effacer le LCD....

Bref pour disposer d'une bibliothèque de fonctions (avec lcdbt.h et lcdbt.c) qu'il suffira d'inclure et de linker dans votre projet pour pouvoir utiliser **de façon structurée** le LCD.

Chaque projet est constitué de 4 grandes parties :

## **I. Une première partie théorique guidée**

Avec au préalable des conseils de lecture pour vous approprier le projet et/ou vous indiquer comment sera réalisé le câblage sur la carte fille multi fonctions fournie. Dans cette partie, vous devrez écrire sur le papier les principales fonctions nécessaires à votre projet (programmation structurée). Vous serez guidé par un « texte à trous ». Ces fonctions seront ensuite utilisées dans un main applicatif (version de base).

## **II. Une deuxième partie intermédiaire**

Qui vous permettra de prendre en main et de tester votre carte multifonctions en lançant des fichiers exécutables tout prêts permettant de vérifier entre autre que le hardware fonctionne correctement.

## **III. Une troisième partie pratique guidée**

Où vous implanterez les fonctions (écrites dans la première partie) sur la carte PICDEM 2 PLUS. Comme on l'a toujours fait en TP et avant d'utiliser ces fonctions dans le main applicatif (version de base) de votre projet, il est **obligatoire** de les tester individuellement. Sinon, vous risquez d'aller dans des impasses et des durées de mise au point très coûteuses en temps...

C'est aussi à ce moment, qu'il faudra penser à structurer votre projet en mettant tout ce qui est lié à votre interface dans des fichiers .h et .C (comme on l'a fait pour le LCD avec lcdbt.h et lcdbt.C). Votre main (c'est-à-dire l'applicatif) sera quant à lui dans un autre fichier .C situé dans un répertoire de même nom et portant le même nom que votre projet (comme on l'a vu en TP).

## **IV. Une dernière partie plus libre et en plus grande autonomie.**

Il s'agira là de mettre en œuvre le LCD pour y afficher des informations liées à votre projet (paramètres de réglages, messages divers...).

Et dans la mesure du temps disponible, on envisagera de coupler des projets :

- par exemple simuler un radar de recul en couplant le projet moteur pas à pas et la mesure de distance,
- le clavier avec les modes de fonctionnement du servomoteur...
- ou tout autre couplage suivant vos éventuelles propositions...

Cette dernière étape sera facilitée si tout a été fait de façon structurée avec des fichiers .h et des fichiers .C.

# **MOTEUR PAS A PAS**

## I. PARTIE THEORIQUE GUIDEE

Avant de démarrer, prenez connaissance des pages 47 à 71 du document :

[Résumé pour les projets PIC.pdf](#)

situé dans le sous répertoire :

[d\\_pic\\_ii/projets/Résumés cours](#)

de votre répertoire de travail.

On se propose, dans ce projet, de réaliser, par étapes successives, un petit programme permettant la commande de vitesse d'un moteur pas à pas, très utilisé dans les imprimantes et les applications liées à la robotique par exemple.

Le moteur pas à pas utilisé (**moteur à 6 fils**) possède 2 **bobinages** appelés **A** (entre 1 et 2) et **B** (entre 3 et 4), chacun possédant son **propre point milieu** (5 et 6) ce qui donne au total 4 « **demi-bobinages** » comme indiqué sur la figure 1. ci-dessous :

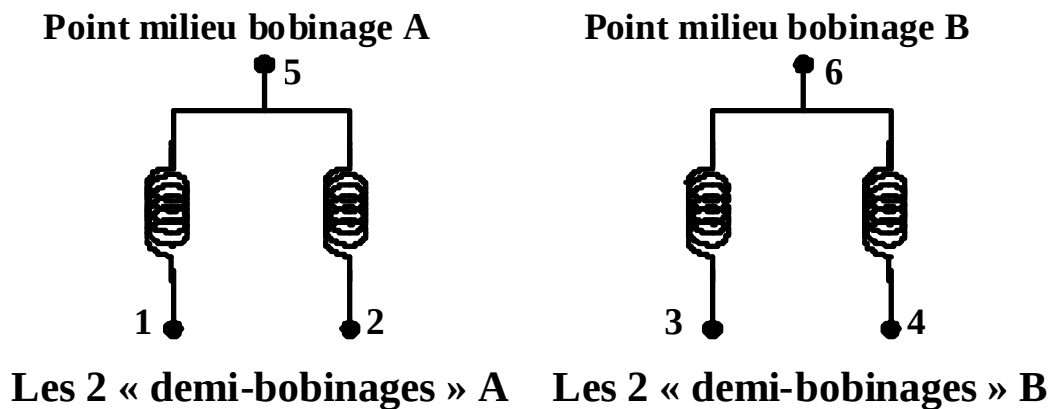


Figure 1.

L'objectif de la commande consiste à faire circuler successivement et alternativement suivant une séquence bien définie (que l'on indiquera plus loin sur la figure 3.) un courant dans chaque « demi-bobinage ».

Ces courants vont générer un champ magnétique **tournant** qui va à son tour entraîner la **rotation d'un aimant permanent** solidaire de l'axe du moteur.

Ainsi, chaque impulsion générée sur chaque « demi-bobinage » va se transformer en une rotation angulaire élémentaire appelée **le pas du moteur**.

Le moteur utilisé dans cet exercice possède **24 pas par tour**, ce qui veut dire qu'il faudra **24 impulsions successives** pour faire effectuer un tour complet à l'axe de rotation du moteur.

Par contre, une broche d'un **PORT** du **PIC**, ne peut pas fournir assez de courant pour chaque « demi-bobinage ». Il faut donc utiliser une petite interface de puissance, comme le montre la figure 2. ci-après. On remarque aussi sur cette figure que les 2 points milieu 5 et 6 des bobinages **A** et **B** sont reliés à une alimentation de puissance **5V PWR** et qu'on utilise les broches **RA1, RA2, RA3 et RA5** du **PORTA** pour piloter le moteur.

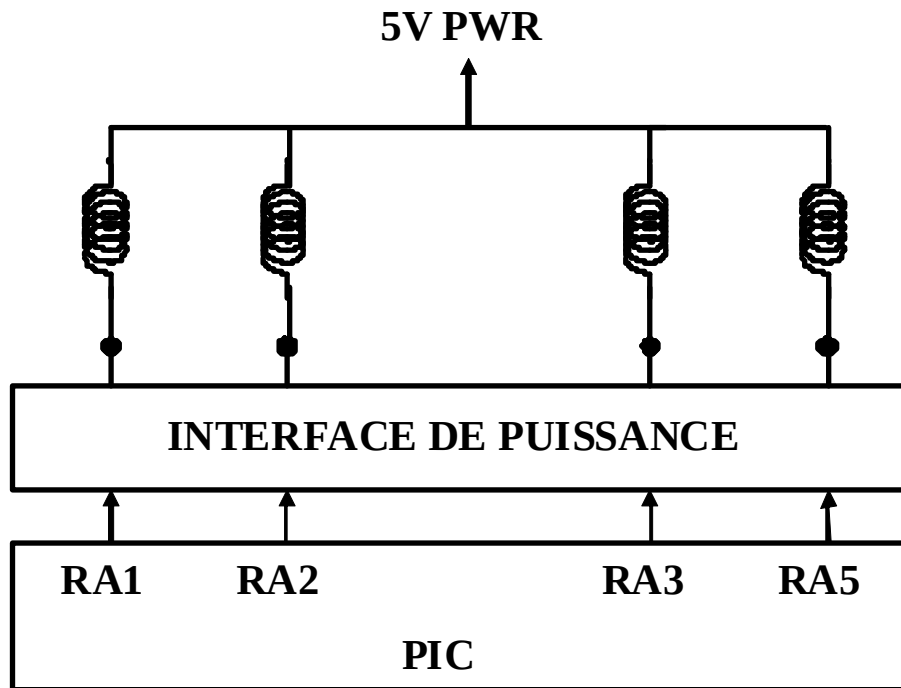


Figure 2.

Pour assurer la rotation du moteur à vitesse constante, il faut respecter, pour les broches RAi, les séquences représentées sur la figure 3. le niveau haut correspondant à 5V et le bas à 0V.

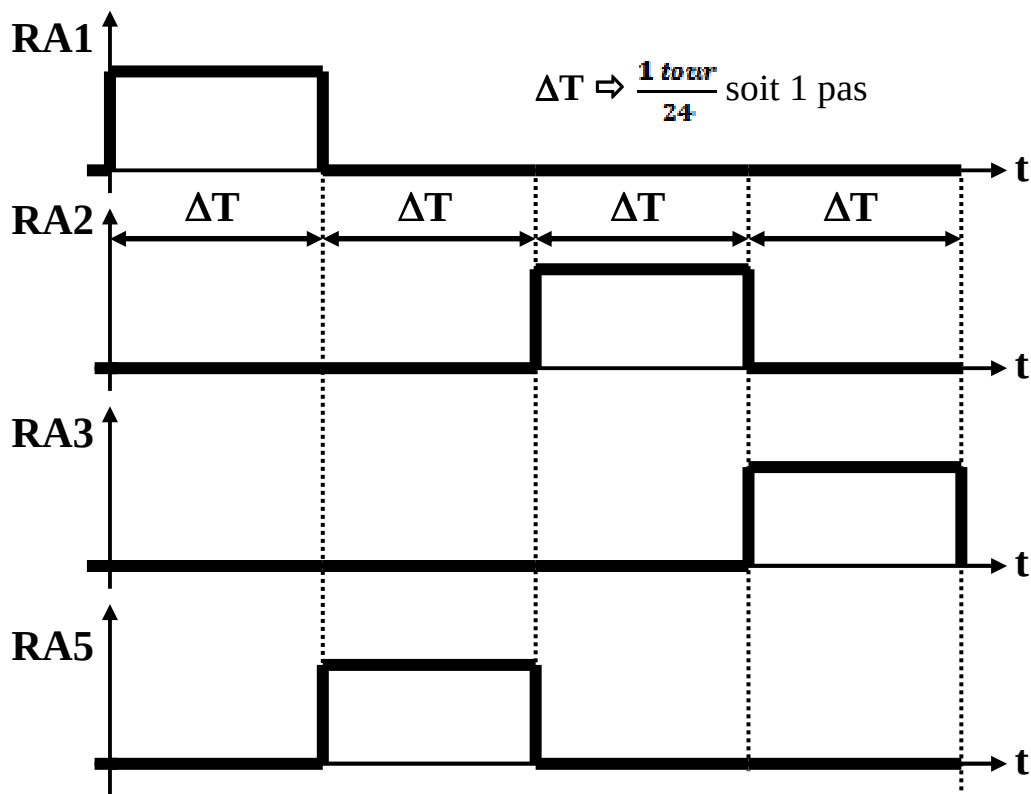


Figure 3.

La durée  $\Delta T$  de chaque impulsion est donc importante, car c'est elle qui fixera la vitesse de rotation  $N$  du moteur.

## 1. Gestion des temporisations.

Les temporisations  $\Delta T$  seront réalisées en utilisant le **TIMER0** du **PIC** (temporisations **matérielles** utilisées en mode scrutation).

On suppose de plus que le **PIC** possède un quartz cadencé à **4 MHz**.

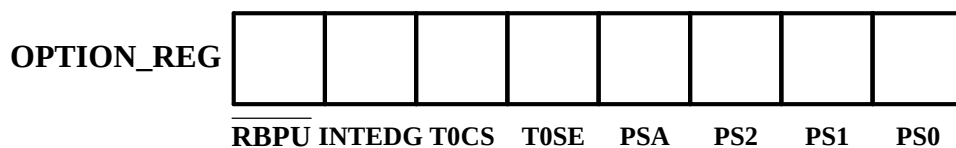
**1.1. Déterminer la Valeur Initiale VI à écrire dans le registre `compteur TMR0` du `TIMER0` pour obtenir une temporisation de **1 ms.** (on prendra la valeur la plus petite pour le prédiviseur p).**

*(mettre ci-dessous les calculs annexes et les justifications de cette valeur).*

[illegible]

**1.2.** En déduire la valeur à écrire dans le registre de **contrôle OPTION\_REG** (reconnu sous le nom **OPTION** par le compilateur C) du **TIMER0** pour obtenir cette temporisation.

*(mettre ci-dessous les justifications de cette valeur).*

[illegible]

This image shows a blank sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

This image shows a blank sheet of white paper with horizontal ruling lines. The lines are evenly spaced and extend across the width of the page. There are no margins, text, or other markings on the paper.

$$N(trs.mn^{-1}) = \frac{2500}{\Delta T(ms)}$$
[illegible]

**1.7.** En déduire la valeur  $\Delta T_{\min}$  pour obtenir une vitesse de rotation de  $N_{\max} = 100 \text{ trs.mn}^{-1}$ , vitesse **maximum** du moteur.

---

---

---

---

---

---

---

---

---

---

**1.8.** Voyez-vous une explication pour justifier l'existence de cette vitesse maximum ?

---

---

---

---

---

---

---

---

---

---

## **2. Gestion du PORTA.**

Ecrire la fonction void **init\_PORTA** (void) permettant :

- de choisir la **directionnalité** des broches **RA1, RA2, RA3 et RA5** (**et elles-seules !!!**) du **PORTA**.
- et **d'imposer le moteur à l'arrêt** au départ.

---

---

---

---

---

---

---

---

### 3. Pilotage à vitesse constante.

En utilisant à bon escient les fonctions précédemment écrites, réaliser un petit programme applicatif permettant **de faire tourner indéfiniment** le moteur à une vitesse de :

$$N = \frac{N_{max}}{2} = 50 \text{ tr.s.mn}^{-1}.$$

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.



On fera travailler le **CAN** :

- (mettre ci-dessous les justifications de cette valeur).*

This image shows a blank sheet of white paper with horizontal blue ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

|      |  |  |  |       |       |       |       |
|------|--|--|--|-------|-------|-------|-------|
|      |  |  |  |       |       |       |       |
| ADFM |  |  |  | PCFG3 | PCFG2 | PCFG1 | PCFG0 |

[illegible]

**4.1.4.** Ecrire ensuite la fonction unsigned short **conversion** (void) qui effectue la **conversion analogique/numérique** de  $V_C$  et qui renvoie en retour sa **valeur numérique**  $V_C^*$  codée sur **16 bits** (on utilisera la technique du « **pooling** »).

[illegible]

On souhaite obtenir la correspondance suivante, entre la consigne  $V_C$  (ou  $V_C^*$ ) et la vitesse  $N$  de rotation du moteur, c'est-à-dire une relation **linéaire**.

This image shows a single sheet of white paper with horizontal blue or grey ruling lines. The lines are evenly spaced and run across the width of the page. There are approximately 20 lines visible. The paper has a slight shadow on the right side, suggesting it's resting on a surface.[illegible]

### 4.3. Synthèse.

Modifier en conséquence votre programme applicatif de la question 3. pour réaliser la commande du moteur pas à pas à **vitesse variable**.

[illegible]

## 5. Critique des résultats obtenus avec cette méthode.

### 5.1. Quel est à votre avis le principal inconvénient de cette petite application ?

**Aide :** on pourra raisonner aux très basses vitesses de rotation obtenues pour des faibles valeurs de la consigne

This image shows a blank sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

**5.2.** Proposez une méthode ou tout au moins un principe pour s'affranchir de ce problème.

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

**ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!**

## **II. TESTS DU HARDWARE CARTE MULTIFONCTIONS**

Il est temps de consulter le schéma d'implantation de la carte multi fonctions situé dans votre répertoire de travail et dans :

**d\_pic\_ii/projets/documentations projets/docs carte multi fonction.**

Prenez connaissance de ce schéma d'implantation.

### **IMPORTANT**

**Avant de relier le moteur pas à pas à la carte multi fonctions et de la mettre sous tension, voir avec l'enseignant pour valider le câblage.**

Pour avoir une idée sur ce qu'on demande de faire, sous **MPLAB** faire :

**File Import...** et aller chercher **dans votre répertoire** de travail dans le sous répertoire :

**d\_pic\_ii/projets/fichiers HEX OK projets**

le fichier : **moteur pas a pas RA0.hex** (c'est un exécutable)

Puis, choisir éventuellement l'outil **MPLAB ICD3** (par **Debugger Select Tool**), envoyer cet exécutable sur l'ICD3 (par **Debugger Program**), et lancer ce programme...

**ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!**

### **III. PARTIE PRATIQUE GUIDEE**

Vous pouvez maintenant implanter les fonctions écrites dans la partie théorique guidée I précédente et les tester individuellement, quand c'est possible.

Puis, implantez le main (version de base) de la partie théorique et testez le.

Pensez aussi dans cette partie, à structurer votre projet en mettant ce qui est lié à votre périphérique :

- ce qui est lié au hardware et les prototypes de fonctions dans un fichier .h
- le code des fonctions dans un fichier .C

**ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!**

## **IV. PARTIE LIBRE EN PLUS GRANDE AUTONOMIE**

Utilisez le LCD pour afficher en temps réel la vitesse de rotation (en trs/min) du moteur.

Couplez votre projet avec celui du télémètre à ultra-sons pour réaliser l'équivalent d'un radar de recul...

En accord avec l'enseignant, on pourra envisager d'autres type de couplage, comme par exemple avec le projet clavier :

Par exemple, sous **MPLAB** faire :

**File**      **Import...**      et aller chercher **dans votre répertoire** de travail dans le sous répertoire :

**d\_pic\_ii/projets/fichiers HEX OK projets**

le fichier :      **moteur pas a pas clavier.hex**      (c'est un exécutable)

Puis, choisir éventuellement l'outil **MPLAB ICD3** (par **Debugger**      **Select Tool**), envoyer cet exécutable sur l'ICD3 (par **Debugger**      **Program**), et lancer ce programme...

**ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!**

**TELEMETRE A**

**ULTRA SONS**

# I. PARTIE THEORIQUE GUIDEE

Avant de démarrer, prenez connaissance :

- des pages 41 à 46 (pour la partie émetteur/récepteur ultra sons) du document :

Résumé pour les projets PIC.pdf

- des pages 108 à 115 (pour la fonction Input Capture du PIC) du document :

Résumé du cours pendant TP PIC.pdf

situés tous les deux dans le sous répertoire :

d\_pic\_ii/projets/Résumés cours

de votre répertoire de travail.

On se propose dans ce projet de réaliser la mesure d'une **distance d** en utilisant :

- un **émetteur** d'ultrasons **E**,
- un **récepteur** d'ultrasons **R**,

comme indiqué sur la figure 1. ci-dessous :

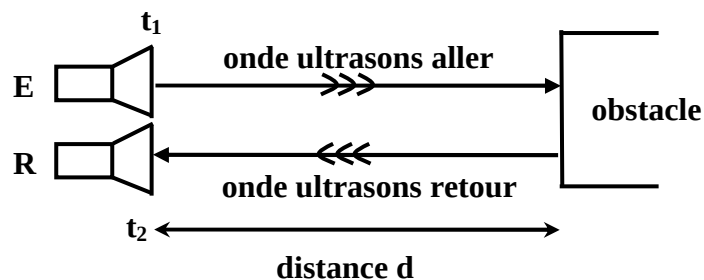


Figure 1.

L'onde sonore émise par **E** à un instant  $t_1$  se propage dans l'air à la vitesse **V** (que l'on supposera ici constante), se réfléchit sur un obstacle situé à la distance **d** de **E** et revient « en écho » vers le récepteur **R** qui la détecte légèrement plus tard à un instant  $t_2$ .

L'idée de base, pour déterminer la distance **d**, est de mesurer tout d'abord la **durée d'aller/retour** de l'onde (appelé aussi **temps de vol**)  $\Delta T = t_2 - t_1$  puis d'en déduire ensuite **d**.

## 1. Un peu de physique...

1.1. Donner l'expression de **d** en fonction de  $\Delta T$  et de **V**.

---

---

---

---

---

---

Les équations de propagation des ondes sonores dans l'air associées à celles de la thermodynamique des gaz montrent qu'une expression (approchée) de la vitesse **V** de l'onde sonore est donnée, en **m.s<sup>-1</sup>**, par :

$$V \approx 332 + 0,6.\theta$$

où **θ** représente la température de l'air en **°C**.

**1.2.** Calculer la vitesse **V** de l'air à la température ambiante de **20°C**.

---

---

---

---

---

---

---

---

---

---

**1.3.** En déduire que l'on peut écrire :

$$d \approx 0,0172. \Delta T$$

avec **d** exprimé en **cm** et **ΔT** exprimée en **μs**.

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

## 2. Un peu de hardware...

Comme souvent en électronique, on va utiliser une **interface extérieure IEUS** qui va s'occuper de l'**émission** (en générant une salve d'ultrasons) et de la **réception** de ces ultrasons. C'est cette interface que l'on aperçoit sur la figure 2. ci-dessous :

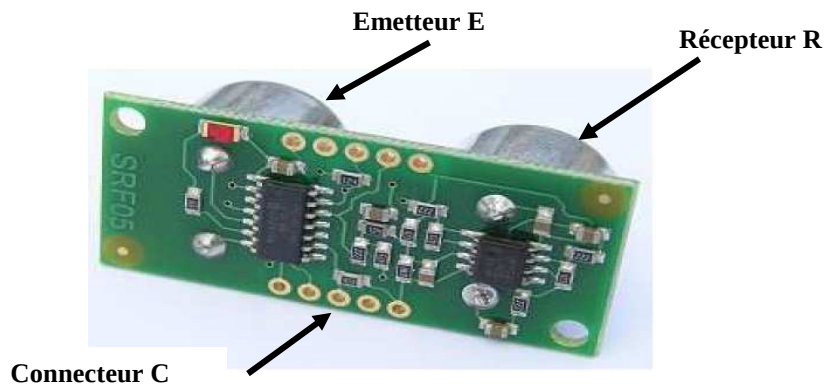


Figure 2.

Le connecteur **C** de l'interface **IEUS**, constitué de **5 points** (dont 4 sont réellement utilisés car la broche 2 n'est pas connectée) sera utilisé pour la liaison avec le **PORTC** d'un **PIC 16F877** cadencé à **4 MHz** comme indiqué sur la figure 3.

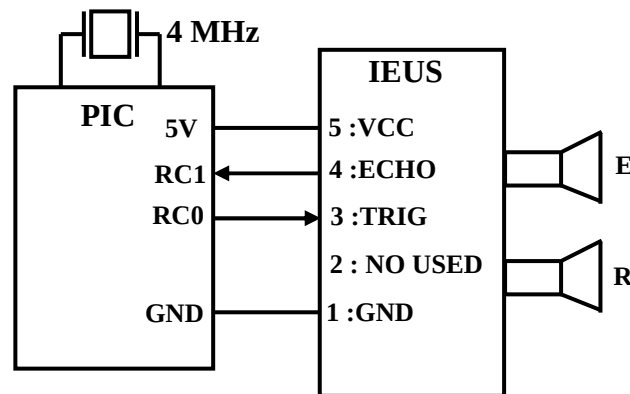


Figure 3.

Le fonctionnement, assez simple, est décrit par les chronogrammes de la figure 4. ci-dessous :

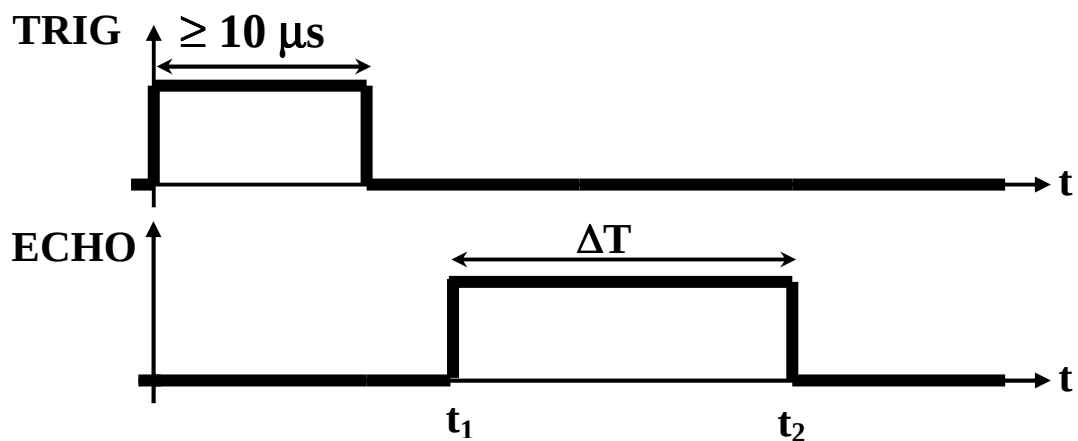


Figure 4.

**La sortie** ECHO de IEUS passera alors **automatiquement** au niveau **haut** à l'instant effectif  $t_1$  de l'émission et repassera **automatiquement** au niveau **bas** à l'instant  $t_2$  de la réception par R de l'onde réfléchie.

En fonction des affectations qui ont été faites, écrire la fonction void **init\_PORTC** (void) permettant de choisir la **directionnalité** des broches **RC0** et **RC1** (**et elles-seules !!!**) du **PORTC**.

[illegible]

Le constructeur de l'interface **IEUS** indique, dans sa documentation, que la distance **d (en cm)** de l'obstacle sera obtenue à partir de la durée  **$\Delta T$  (en  $\mu s$ )** par la relation :

Montrer que cette relation est conforme à celle obtenue à la question **1.3.** précédente.

[illegible]

### 3. Un peu de software...

#### 3.1. Commande de l'interface IEUS.

##### 3.1.1. Temporisation.

On suppose que la « boucle for » suivante, mesurée sous simulateur, dure **1 ms** pour la constante **tempo** valant **200**.

```
char i;
```

```
for (i=1; i < tempo; i++);
```

En déduire, en supposant une proportionnalité entre la durée de cette boucle et tempo, la nouvelle valeur de la constante tempo pour obtenir une durée de boucle compatible avec la durée du pulse sur l'entrée **TRIG**.

---

---

---

---

---

---

---

---

---

---

##### 3.1.2. Génération du pulse sur l'entrée TRIG.

Ecrire la fonction void **pulse** (void) qui génère un pulse de commande de l'**IEUS** conforme aux chronogrammes de la figure 4. précédente.

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

### 3.2. Mesure de $\Delta T$ .

#### 3.2.1. Principe de la mesure.

La mesure du temps de vol  $\Delta T$  se fera en utilisant la fonction **INPUT CAPTURE** du module **CCP2** du **PIC**.

##### 3.2.1.1. Justifier ce choix.

---

---

---

---

---

---

---

---

---

---

3.2.1.2. Indiquez la valeur des différents bits du registre **CCPCON2** permettant d'initialiser correctement le module **CCP2** du **PIC** après exécution de la fonction `pulse()` précédente. *(mettre ci-dessous les justifications des valeurs des différents bits de **CCPCON2**).*

---

---

---

---

---

---

---

---

---

---

---

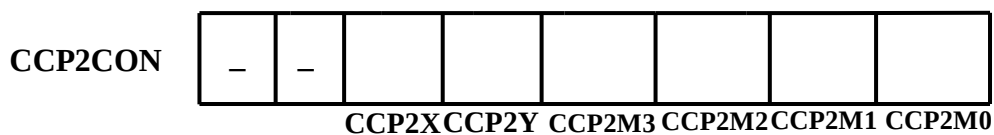
---

---

---

---

---



3.2.1.3. Quel timer interne est utilisé par le **PIC** pour réaliser la référence temporelle de la fonction **INPUT CAPTURE** ?

---

---

---

---

---

(mettre ci-dessous les justifications des valeurs des différents bits de **T1CON**).

This image shows a blank sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

|       |         |         |         |        |        |        |  |
|-------|---------|---------|---------|--------|--------|--------|--|
| T1CON | —       | —       |         |        |        |        |  |
|       | T1CKPS1 | T1CKPS0 | T1OSCEN | T1SYNC | TMR1CS | TMR1ON |  |

This image shows a single sheet of white paper with horizontal blue ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

### **3.2.2. Fonctionnement sous interruption.**

**Aide** : on veut générer une interruption au **premier front montant** de la sortie **ECHO**, ce qui permettra de mesurer l'instant **t<sub>1</sub>**.

Mais, il faudra aussi penser, dans la routine d'interruption, à re initialiser le registre **CCP2CON** pour être sensible sur le **prochain front descendant** de cette même sortie

**ECHO**, ce qui permettra de mesurer  $t_2$ .

Et d'en déduire  $\Delta T$  !!!

**3.2.2.1. Indiquez les différents masques d'interruption à mettre en œuvre et leurs valeurs d'initialisation.**

[illegible]

### 3.2.2.2. Ecrire la routine d'interruption permettant de mesurer le temps de vol $\Delta T$ .

This image shows a blank sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

#### 4. Synthèse de l'application...

Regrouper votre travail et réaliser un programme principal permettant de mesurer la distance d.

This image shows a single page of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

**ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!**

## **II. TESTS DU HARDWARE CARTE MULTIFONCTIONS**

Il est temps de consulter le schéma d'implantation de la carte multi fonctions situé dans votre répertoire de travail et dans :

[d\\_pic\\_ii/projets/documentations projets/docs carte multi fonction.](#)

Prenez connaissance de ce schéma d'implantation.

### **IMPORTANT**

**Avant de relier l'émetteur récepteur ultra sons à la carte multi fonctions et de la mettre sous tension, voir avec l'enseignant pour valider le câblage.**

Pour avoir une idée sur ce qu'on demande de faire, sous **MPLAB** faire :

**File Import...** et aller chercher **dans votre répertoire** de travail dans le sous répertoire :

[d\\_pic\\_ii/projets/fichiers HEX OK projets](#)

le fichier : **telemetre lcd ascii.hex** (c'est un exécutable)

Puis, choisir éventuellement l'outil **MPLAB ICD3** (par **Debugger Select Tool**), envoyer cet exécutable sur l'ICD3 (par **Debugger Program**), et lancer ce programme...

|                                                                    |
|--------------------------------------------------------------------|
| <b>ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!</b> |
|--------------------------------------------------------------------|

### **III. PARTIE PRATIQUE GUIDEE**

Vous pouvez maintenant implanter les fonctions écrites dans la partie théorique guidée I précédente et les tester individuellement, quand c'est possible.

Puis, implantez le main (version de base) de la partie théorique et testez le.

Pensez aussi dans cette partie, à structurer votre projet en mettant ce qui est lié à votre périphérique :

- ce qui est lié au hardware et les prototypes de fonctions dans un fichier .h
- le code des fonctions dans un fichier .C

**ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!**

## **IV. PARTIE LIBRE EN PLUS GRANDE AUTONOMIE**

Utilisez le LCD pour afficher en temps réel la distance mesurée (en cm) de l'obstacle.

Couplez votre projet avec celui du moteur pas à pas pour réaliser l'équivalent d'un radar de recul...

En accord avec l'enseignant, on pourra envisager d'autres type de couplage à définir.

**ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!**

# **SERVOMOTEUR**

## I. PARTIE THEORIQUE GUIDEE

Avant de démarrer, prenez connaissance des pages 103 à 118 du document :

[Résumé pour les projets PIC.pdf](#)

situé dans le sous répertoire :

[d\\_pic\\_ii/projets/Résumés cours](#)

de votre répertoire de travail.

On se propose dans ce projet de réaliser, par étape, un petit programme permettant de réaliser la commande d'un **servomoteur**.

Un **servomoteur** est en général constitué d'un **petit moteur à courant continu** auquel a été rajouté un **bloc électronique** permettant d'assurer un **asservissement de la position** de ce moteur.

Cela signifie qu'on peut imposer un angle de rotation (**consigne**) à ce moteur : le moteur **tourne alors de cet angle (phase dynamique transitoire)** et **se maintient ensuite dans cette nouvelle position (phase statique permanente)** même s'il est soumis à un couple résistant (position la plus **stable** possible), et cela jusqu'à la prochaine modification de la consigne.

Le servomoteur mis en œuvre dans cet exercice possède **3 bornes (1, 2 et 3)**, comme indiqué sur la figure 1. ci-dessous :

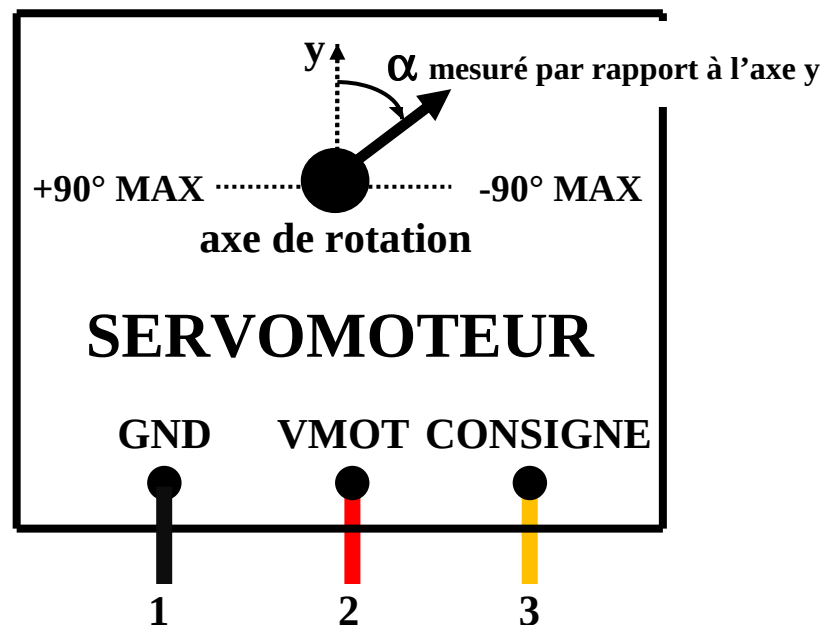


Figure 1.

On utilisera une **alimentation de puissance de 5 V**, branchée entre les bornes **1 (GND)** et **2 (VMOT)**, pour alimenter le servomoteur.

La valeur de la consigne est fournie sur la borne **d'entrée 3**, appelée **CONSIGNE**, sous la forme d'un signal **PWM** dont les variations sont représentées sur la figure 2. ci-dessous :

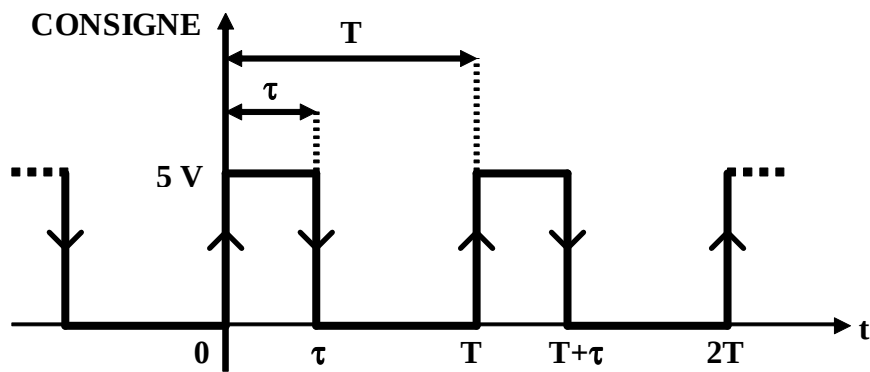


Figure 2.

La valeur de **T** est imposée à **20 ms** par le constructeur, et le lien entre **tau** (durée à l'état haut du signal **PWM** et  $< T$ ) et **alpha** (angle de rotation souhaité mesuré par rapport à l'axe **y** comme indiqué sur la figure 1. précédente) est fourni par la courbe de la figure 3. suivante :

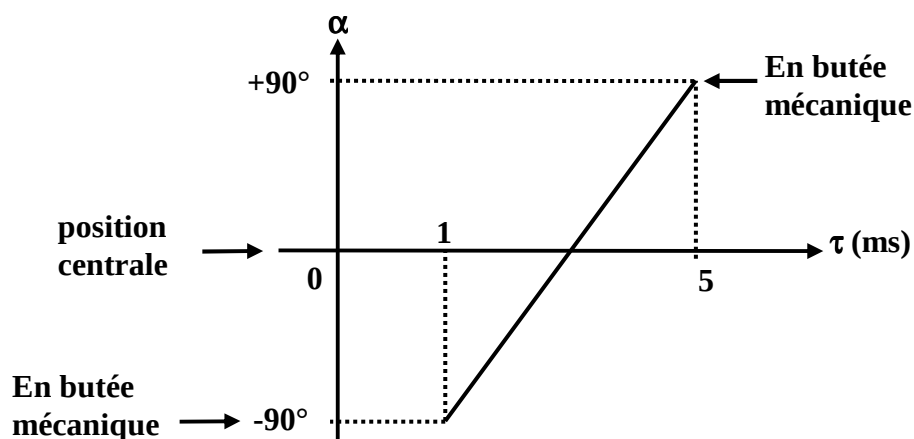


Figure 3.

Remarquons que pour éviter de tourner **au-delà de +90°** ou **en-deçà de -90°**, un système de **butées mécaniques** (sécurité) bloque la rotation du servomoteur.

### 1. Définition de quelques constantes et mise en place de butées logicielles

Définir par les directives adéquates les constantes **T**, **tau\_min** et **tau\_max** (qui joueront le rôle de **butées logicielles** pour éviter de solliciter les butées mécaniques), toutes ces constantes étant exprimées en **ms**.

---

---

---

---

---

---

---

## 2. Gestion des temporisations.

**Remarque importante** : même si cette solution était possible ici, on n'utilisera pas dans cet exercice le mode **PWM** des modules **CPP1** ou **CCP2** du **PIC**.

Dans un premier temps, les durées **T** et  **$\tau$**  seront réalisées en utilisant le **TIMER0** du **PIC** (temporisations **matérielles** utilisées en mode scrutation).

On suppose de plus que le **PIC** possède un quartz cadencé à **4 MHz**.

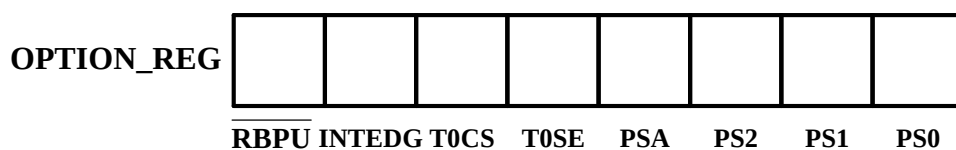
**2.1. Déterminer la Valeur Initiale VI** à écrire dans le registre **compteur TMR0** du **TIMER0** pour obtenir une temporisation de **1 ms.** (on prendra la valeur la plus petite pour le **prédiviseur p**).

*(mettre ci-dessous les calculs annexes et les justifications de cette valeur).*

[illegible]

**2.2.** En déduire la valeur à écrire dans le registre de **contrôle OPTION\_REG** (reconnu sous le nom **OPTION** par le compilateur C) du **TIMER0** pour obtenir cette temporisation.

*(mettre ci-dessous les justifications de cette valeur).*

[illegible]

This image shows a blank sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

[illegible]
$$\alpha(^{\circ}) = 45. \tau(\text{ms}) - 135$$
[illegible]

---

---

---

---

---

---

### **3. Gestion du PORTC.**

L'entrée **CONSIGNE** du servomoteur est reliée sur la broche **RC2** du **PORTC** du **PIC**.

Ecrire la fonction void **init\_PORTC** (void) permettant :

- de choisir la **directionnalité** de la broche **RC2** (**et elle-seule !!!**) du **PORTC**.
- et **d'imposer aucune consigne d'angle** au départ.

---

---

---

---

---

---

---

---

### **4. Asservissement à angle de rotation constant.**

En utilisant à bon escient les fonctions précédemment écrites, réaliser un petit programme applicatif permettant d'asservir **le servomoteur avec un angle  $\alpha_0 = +45^\circ$** .

**N'oubliez pas de gérer les butées logicielles, même si dans ce cas ce n'est pas obligatoire puisque l'angle de rotation est dans l'intervalle [ -90° ; +90 ] !!!**

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

### **5. Asservissement sur une consigne extérieure réglable.**

Pour **pouvoir modifier l'angle de rotation** du servomoteur en imposant **une consigne d'angle**, on va utiliser un potentiomètre **P** branché sur la patte **RA0** du **PIC**, comme indiqué sur la figure 4. ci-dessous.

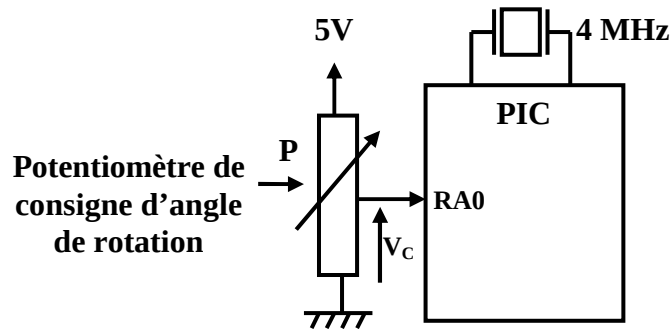


Figure 4.

### **5.1 Etude de la Conversion Analogique Numérique.**

**5.1.1. Déterminer la valeur à écrire dans le registre de **contrôle ADCON0** du CAN.**  
*(mettre ci-dessous les justifications de cette valeur).*

This image shows a blank sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

|               |       |       |      |      |      |                           |        |
|---------------|-------|-------|------|------|------|---------------------------|--------|
| <b>ADCON0</b> |       |       |      |      |      |                           |        |
|               | ADCS1 | ADCS0 | CHS2 | CHS1 | CHS0 | <u>GO/</u><br><u>DONE</u> | - ADON |

On fera travailler le **CAN** :

- (mettre ci-dessous les justifications de cette valeur).*

[illegible]

|      |  |  |  |       |       |       |       |
|------|--|--|--|-------|-------|-------|-------|
|      |  |  |  |       |       |       |       |
| ADFM |  |  |  | PCFG3 | PCFG2 | PCFG1 | PCFG0 |

[illegible]

**5.1.4.** Ecrire ensuite la fonction unsigned char **conversion** (void) qui effectue la **conversion analogique/numérique** de  $V_C$  et qui renvoie en retour sa **valeur numérique**  $V_C^*$  codée sur **8 bits** (on utilisera la technique du « **pooling** »).

[illegible]

## 5.2. Détermination de la largeur des impulsions de commande du signal de consigne.

On souhaite obtenir la correspondance suivante, entre la consigne  $V_C$  (ou  $V_C^*$ ) et l'angle de rotation  $\alpha$  du servomoteur, c'est-à-dire une relation **affine**, comme indiqué sur la figure 5.

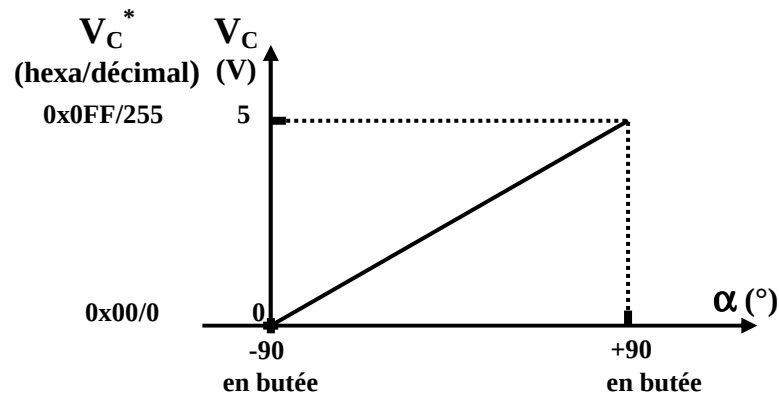


Figure 5.

5.2.1. Donner l'expression de l'angle de rotation  $\alpha$  (en °) stocké dans la variable **alpha** en fonction de  $V_C^*$ .

---

---

---

---

---

---

---

---

---

---

---

---

5.2.2. En déduire la **durée à l'état haut**, stockée dans la variable **tau\_en\_ms** (en ms) du signal **PWM** à générer pour pouvoir faire varier l'angle de rotation du servomoteur en fonction du réglage du potentiomètre **P** de consigne position.

---

---

---

---

---

---

---

---

---

---

---

---

### 5.3. Synthèse.

Modifier en conséquence votre programme applicatif de la question 4. pour réaliser la commande de la position du servomoteur par le potentiomètre de consigne.

[illegible]

## **6. Critiques des résultats obtenus avec cette méthode.**

### 6.1. Critique 1 : base de temps utilisée

**6.1.1.** Compte tenu de la **granularité des temporisations** utilisées et du **type** de la variable **tau\_en\_ms**, quelles seront les valeurs possibles pour les angles de rotation du servomoteur ?

This image shows a blank sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

**6.1.2.** Proposez une méthode ou tout au moins un principe pour obtenir un réglage « **plus fin et plus précis** » de la position du servomoteur.

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

## **6.2. Critique 2 : évolutivité vers un système plus complexe**

### 6.2.1. Dans un projet de robotique plus complet, le PIC peut être amené à gérer **un nombre de capteurs et d'actionneurs beaucoup plus important.**

Expliquer alors pourquoi le programme applicatif précédent pose un problème d'évolutivité vers cette augmentation de la complexité.

[illegible]

**6.2.2.** Proposez une méthode ou tout au moins un principe pour permettre cette évolution.

This image shows a full page of white paper with horizontal blue or grey ruling lines. The lines are evenly spaced and run across the width of the page, typical of notebook paper. There are no margins, text, or other markings on the page.

**ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!**

## **II. TESTS DU HARDWARE CARTE MULTIFONCTIONS**

Il est temps de consulter le schéma d'implantation de la carte multi fonctions situé dans votre répertoire de travail et dans :

[d\\_pic\\_ii/projets/documentations projets/docs carte multi fonction.](#)

Prenez connaissance de ce schéma d'implantation.

### **IMPORTANT**

**Avant de relier le servomoteur à la carte multi fonctions et de la mettre sous tension, voir avec l'enseignant pour valider le câblage.**

Pour avoir une idée sur ce qu'on demande de faire, sous **MPLAB** faire :

**File Import...** et aller chercher **dans votre répertoire** de travail dans le sous répertoire :

[d\\_pic\\_ii/projets/fichiers HEX OK projets](#)

le fichier : **servomoteur lcd ascii.hex** (c'est un exécutable)

Puis, choisir éventuellement l'outil **MPLAB ICD3** (par **Debugger Select Tool**), envoyer cet exécutable sur l'ICD3 (par **Debugger Program**), et lancer ce programme...

|                                                                    |
|--------------------------------------------------------------------|
| <b>ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!</b> |
|--------------------------------------------------------------------|

### **III. PARTIE PRATIQUE GUIDEE**

Vous pouvez maintenant implanter les fonctions écrites dans la partie théorique guidée I précédente et les tester individuellement, quand c'est possible.

Puis, implantez le main (version de base) de la partie théorique et testez le.

Pensez aussi dans cette partie, à structurer votre projet en mettant ce qui est lié à votre périphérique :

- ce qui est lié au hardware et les prototypes de fonctions dans un fichier .h
- le code des fonctions dans un fichier .C

**ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!**

#### **IV. PARTIE LIBRE EN PLUS GRANDE AUTONOMIE**

Utilisez le LCD pour afficher par exemple les modes de fonctionnement du servomoteur..

En accord avec l'enseignant, on pourra envisager un couplage avec le projet clavier par exemple.

**ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!**

# **CLAVIER MATRICIEL**

# I. PARTIE THEORIQUE GUIDEE

Avant de démarrer, prenez connaissance des pages 14 à 26 du document :

[Résumé pour les projets PIC.pdf](#)

situé dans le sous répertoire :

[d\\_pic\\_ii/projets/Résumés cours](#)

de votre répertoire de travail.

On se propose dans ce projet de réaliser, étape par étape, un petit programme permettant d'utiliser un clavier branché sur un microcontrôleur PIC.

## 1. Généralités sur les claviers

Les plus simples possèdent des touches (bouton poussoir **BP**) à point commun **PC**, chacune de ces touches étant conforme au schéma de la figure 1.

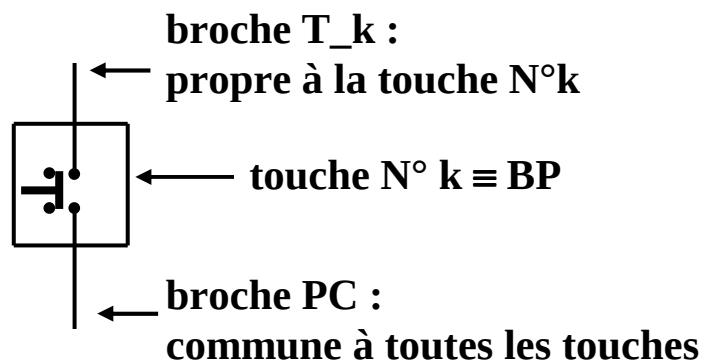


Figure 1.

Toutes les broches **PC** des différentes touches du clavier sont reliées entre elles.

**1.1.** Pour implanter un clavier à point commun de 16 touches, combien de broches d'entrée faut-il mobiliser sur les ports logiques du PIC ?

---

---

---

---

---

---

---

---

Mais il existe aussi des claviers matriciels, comme on peut le voir sur la figure 2. pour un clavier 9 touches, possédant  $n = 3$  lignes et  $p = 3$  colonnes.

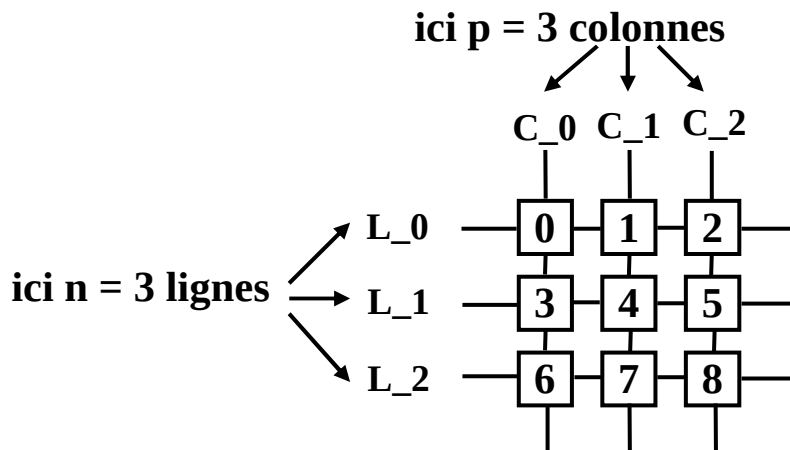


Figure 2.

Chaque **touche N°k** est aussi un **BP** qui, enfoncé, assurera la liaison entre :

- la ligne  $L_i$  où se trouve cette touche N°k et
- la colonne  $C_j$  où se trouve cette touche N°k. (cf. figure 3.).

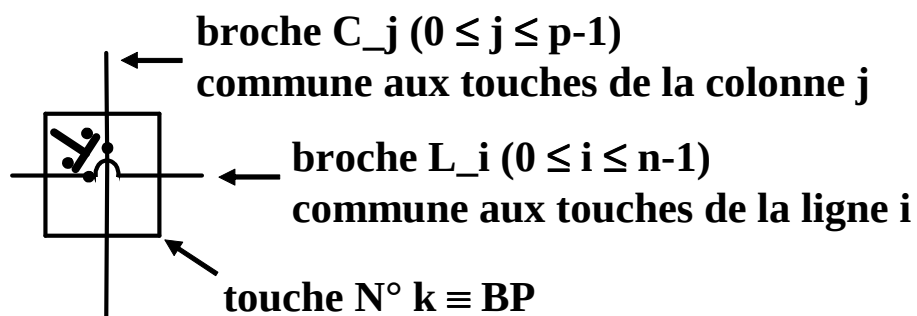


Figure 3.

**1.2.** Pour implanter un clavier matriciel de 16 touches, combien faut-il de lignes et de colonnes pour minimiser le nombre de broches nécessaires sur les ports logiques du PIC ?

---

---

---

---

---

---

---

---

**1.3.** Conclusions sur ces deux types de clavier.

---

---

---

On va utiliser le **PORTB** du PIC pour implanter un clavier matriciel de 16 touches (correspondant aux 16 chiffres hexadécimaux de **0** à **F**) constitué de 4 lignes et de 4 colonnes comme indiqué sur la figure 4.

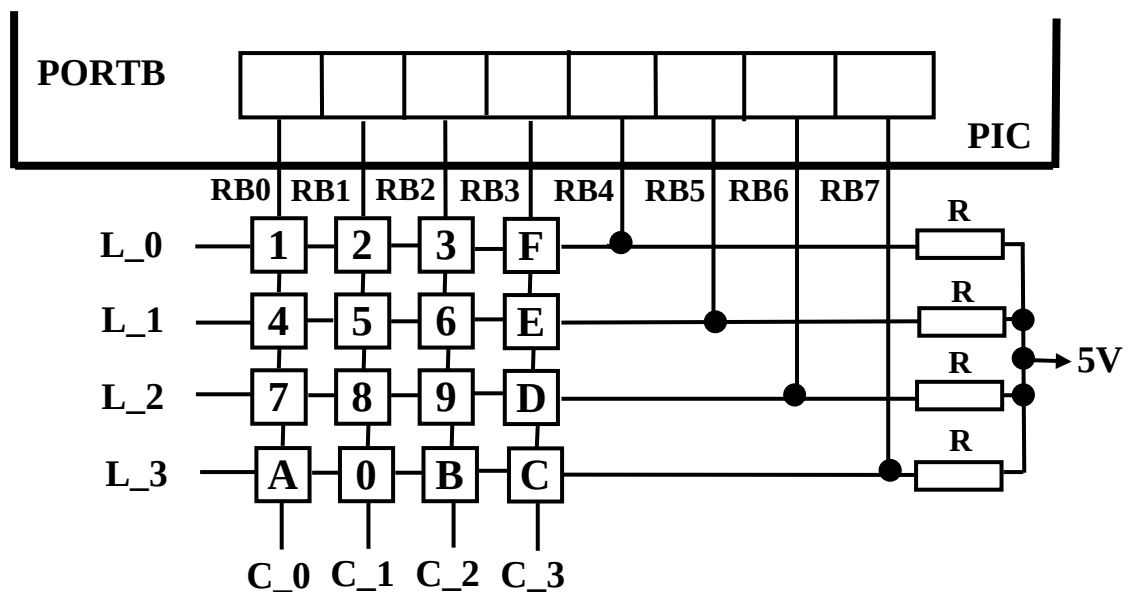


Figure 4.

La détection de l'appui sur une touche est plus compliquée que dans le cas d'un clavier à point commun. Il faut faire ce qu'on appelle un **scanning** (ou **scrutation**, ou multiplexage temporel sur les colonnes).

Les broches **RB3** à **RB0** (donc les **colonnes**) sont mises en **sortie**.

Les broches **RB7** à **RB4** (donc les **lignes**) sont mises en **entrée**.

Voici le principe général de cette scrutation du clavier :

① on active la **colonne 0** en sortant respectivement la valeur binaire **1 1 1 0** respectivement sur les sorties **RB3** à **RB0**,

② on lit ensuite les bits **RB7** à **RB4** de **PORTB** :

- si une des lignes **RB7** à **RB4** est à **0**, c'est qu'on a appuyé sur la touche se trouvant à l'intersection de la **colonne 0** et de cette ligne. Ce qui permet de trouver le N° de la touche enfoncée et d'arrêter la scrutation,

- si toutes les lignes **RB7** à **RB4** sont à **1**, cela signifie qu'aucune des touches de la **colonne 0** n'a été enfoncée. Alors on passe à la colonne suivante (en sortant respectivement la valeur binaire **1 1 0 1** sur les sorties **RB3** à **RB0**) pour scanner la **colonne 1**. Et on recommence en ②. Et ceci jusqu'à la **colonne 3**.

Si à la fin de la scrutation des 4 colonnes, aucune des entrée **RB7** à **RB4** n'est passée à **0**, c'est qu'aucune touche n'a été enfoncée.

## 2. Affectations hardware : un fichier .h

**2.1.** Compte-tenu du principe précédent, à quoi servent les 4 résistances **R** apparaissant sur le schéma de la figure 4 ?

---

---

---

---

---

---

Pour améliorer la lisibilité des programmes, on va définir des constantes hardware pour repérer lignes et colonnes.

**2.2.** Compléter les affectations suivantes :

```
// pour les colonnes
#define C_0 RB0
#define C_1 ____
#define C_2 ____
#define C_3 ____

// pour les lignes
#define L_0 ____
#define L_1 ____
#define L_2 ____
#define L_3 ____

// pour la directionnalité des colonnes
#define TRIS_C_0 TRISB0
#define TRIS_C_1 ____
#define TRIS_C_2 ____
#define TRIS_C_3 ____

// pour la directionnalité des lignes
#define TRIS_L_0 ____
#define TRIS_L_1 ____
#define TRIS_L_2 ____
#define TRIS_L_3 ____
```

On va enfin définir un tableau permettant de donner le code ASCII de la touche enfoncée.

**2.3.** Compléter le tableau suivant :

// tableau des codes ASCII des touches : 4 premiers éléments pour ligne 0, 4 suivants pour ligne 1 ...

```
const unsigned char code_touche[] = {'1','2', _____};
```

### 3. Définition de quelques fonctions de base

On va écrire quelques fonctions de base qui nous permettront de gérer le clavier.

### 3.1. Initialisation du PORTB

Ecrire une fonction de prototype **void init\_clavier (void)** permettant d'initialiser la directionnalité des broches du **PORTB** comme on l'a vu dans la partie précédente.

Penser aussi, dans cette fonction, à gérer le bit **RBPU** du registre **OPTION\_REG**. (ce bit permet de **s'affranchir** de l'utilisation des résistances extérieures R de pull up de la figure 4 précédente en les « incluant » à l'intérieur du PIC).

Voir pour cela extrait de la documentation MICROCHIP ci-dessous :

Each of the PORTB pins has a weak internal pull-up. A single control bit can turn on all the pull-ups. This is performed by clearing bit **RBPU** (**OPTION\_REG<7>**). The weak pull-up is automatically turned off when the port pin is configured as an output. The pull-ups are disabled on a Power-on Reset.

This image shows a full page of white paper with horizontal blue or grey ruling lines. The lines are evenly spaced and run across the width of the page, providing a template for handwriting practice or general note-taking. There are no margins, text, or other markings on the page.

### 3.2. Touche quelconque enfoncée ? Fonction char kbhit (void)

Ecrire une fonction de prototype **char kbhit (void)** permettant de savoir si une touche quelconque du clavier a été enfoncée.

Cette fonction renverra :

- 0 si aucune touche n'est enfoncée pendant son exécution,
- 1 si une touche a été enfoncée pendant son exécution.

(contrairement à la méthode de scrutation précédente, colonne par colonne, on pourra tester les quatre colonnes en même temps, car ici on n'a pas besoin de connaître précisément la valeur de la touche enfoncée mais seulement de savoir si une touche quelconque a été enfoncée).

This image shows a full page of blank white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page, providing a template for writing or drawing. There are no margins, text, or other markings present.

### 3.3. Fonction char getkey (void)

Ecrire une fonction (**non bloquante**) de prototype **char getkey (void)** qui renvoie :

- le code ASCII de la touche si une touche a été enfoncée,
- et 0x00 sinon.

**Remarque** : non bloquante veut dire que la fonction `getkey` n'attend pas qu'on appuie sur le clavier pour renvoyer le code ASCII... Penser pour cela à utiliser la fonction `kbhit`.

[illegible]

#### 4. Synthèse : mise en œuvre du clavier

On suppose disposer d'un afficheur type LCD par exemple et d'une fonction de prototype **void print\_char (char c)** permettant d'afficher le **char c** sur le LCD.

Ecrire un main qui affiche, en écho sur le LCD tous les caractères frappés au clavier.

This image shows a full page of blank handwriting practice paper. It features multiple sets of horizontal lines spaced evenly down the page. Each set typically consists of three lines: a top blue line, a middle green line, and a bottom red line, providing a guide for letter height and placement. The paper is otherwise completely blank, with no text or markings.

**ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!**

## **II. TESTS DU HARDWARE CARTE MULTIFONCTIONS**

Il est temps de consulter le schéma d'implantation de la carte multi fonctions situé dans votre répertoire de travail et dans :

**d\_pic\_ii/projets/documentations projets/docs carte multi fonction.**

Prenez connaissance de ce schéma d'implantation.

### **IMPORTANT**

**Avant de relier le clavier à la carte multi fonctions et de la mettre sous tension, voir avec l'enseignant pour valider le câblage.**

Pour avoir une idée sur ce qu'on demande de faire, sous **MPLAB** faire :

**File Import...** et aller chercher **dans votre répertoire** de travail dans le sous répertoire :

**d\_pic\_ii/projets/fichiers HEX OK projets**

le fichier : **test clavier ICD3.hex** (c'est un exécutable)

Puis, choisir éventuellement l'outil **MPLAB ICD3** (par **Debugger Select Tool**), envoyer cet exécutable sur l'ICD3 (par **Debugger Program**), et lancer ce programme...

( Ou alors faire la même chose avec le fichier :

**lcd clavier flash.hex** qu'il faut par contre **programmer** dans l'EEPROM du PIC par : **Programmer Select Tool MPLAB ICD3** et envoyer sur l'ICD3 par **Programmer Program**. **Retirer** alors le connecteur **RJ11** et vérifier que votre clavier complet fonctionne)

|                                                                    |
|--------------------------------------------------------------------|
| <b>ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!</b> |
|--------------------------------------------------------------------|

### **III. PARTIE PRATIQUE GUIDEE**

Vous pouvez maintenant implanter les fonctions écrites dans la partie théorique guidée I précédente et les tester individuellement, quand c'est possible.

Puis, implantez le main (version de base) de la partie théorique et testez le.

Pensez aussi dans cette partie, à structurer votre projet en mettant ce qui est lié à votre périphérique :

- ce qui est lié au hardware et les prototypes de fonctions dans un fichier .h
- le code des fonctions dans un fichier .C

**ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!**

## **IV. PARTIE LIBRE EN PLUS GRANDE AUTONOMIE**

Utilisez le LCD pour afficher, en écho, les caractères frappés au clavier

Couplez votre projet avec le servomoteur par exemple pour commander ses différents mode de fonctionnement à partir du clavier.

Ou avec le moteur pas à pas.

Par exemple, sous **MPLAB** faire :

**File**            **Import...**    et aller chercher **dans votre répertoire** de travail  
dans le sous répertoire :

**d\_pic\_ii/projets/fichiers HEX OK projets**

le fichier :    **moteur pas a pas clavier.hex**            (c'est un exécutable)

Puis, choisir éventuellement l'outil **MPLAB ICD3** (par **Debugger**            **Select Tool**),  
envoyer cet exécutable sur l'ICD3 (par **Debugger**    **Program**), et lancer ce programme...

|                                                                    |
|--------------------------------------------------------------------|
| <b>ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!</b> |
|--------------------------------------------------------------------|

**GPS**

## I. PARTIE THEORIQUE GUIDEE

Avant de démarrer, prenez connaissance :

des pages 119 à 140 (pour la partie GPS et liaison série) du document :

[Résumé pour les projets PIC.pdf](#)

situé dans le sous répertoire :

[d\\_pic\\_ii/projets/Résumés cours](#)

de votre répertoire de travail.

On se propose dans ce projet de réaliser, étape par étape, un petit programme permettant de récupérer quelques unes des informations fournies par un module récepteur **GPS** (Global Positionning System) comme :

- l'heure **GMT** (Greenwich Mean Time),
- la date,
- la latitude,
- et la longitude du lieu.

On rappelle que le système **GPS**, développé par les U.S.A. permet, grâce à un réseau de 24 satellites, de se positionner n'importe où dans le monde grâce au procédé de la triangularisation.

Dans ce problème, toutes les informations récupérées des satellites sont traitées en interne par le module récepteur **GPS** utilisé, qui peut ensuite les fournir à l'extérieur (voir page suivante les 2 modes de fonctionnement du récepteur **GPS**) :

- soit de façon systématique et régulière,
- soit à la demande.

Pour communiquer avec l'extérieur (ici ce sera avec un **PIC 16F877A**), le module **GPS** dispose de **4 broches**, comme indiqué sur la figure 1.

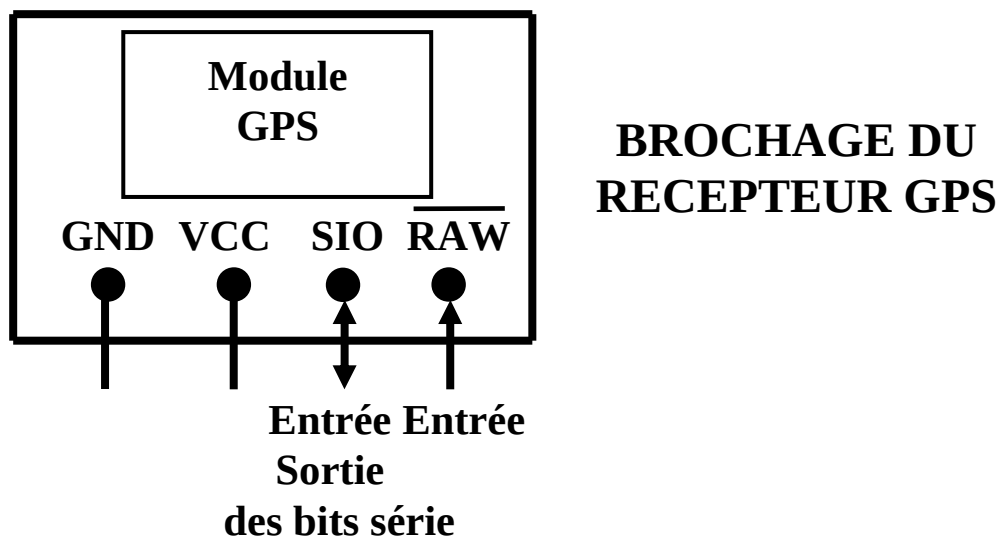


Figure 1.

La communication des informations se fait par la broche **SIO** (Serial Input Output) au format **TTL NRZ série asynchrone** :

- 4800 bauds,
- 1 bit de start,
- 8 bits de données (émission de D0 (LSB) en premier, puis D1, ... jusqu'à D7 (MSB))
- pas de parité,
- 1 bit de stop.

L'entrée  $\overline{RAW}$  permet de choisir entre 2 modes de fonctionnement :

↪ si l'entrée  $\overline{RAW} = 0$  : mode dit **RAW**.

Le récepteur **GPS** envoie en permanence et régulièrement des trames d'octets sous forme série sur sa broche **SIO** qui fonctionne alors uniquement en sortie.

Ces trames sont formatées et contiennent toutes les informations comme la position, l'heure, la date, le nombre de satellites en vue...

↪ si l'entrée  $\overline{RAW} = 1$  : mode dit **SMART**.

Le récepteur répond (sur sa broche **SIO**) aux requêtes (ou commandes) reçues sur cette même broche **SIO** (protocole type question/réponse).

La broche **SIO** devient alors **bidirectionnelle** :

- tantôt en **entrée** au moment de la réception de la commande par le module **GPS**,
- tantôt en **sortie** pour la réponse à cette commande de la part du module **GPS**.

C'est le mode **SMART** qui sera mis en œuvre dans cet exercice.

Le synoptique d'implantation de ce module **GPS** sur une **carte d'extension** connectée à une **carte mère** comportant un **PIC 16F877A** est fourni sur la figure 2.

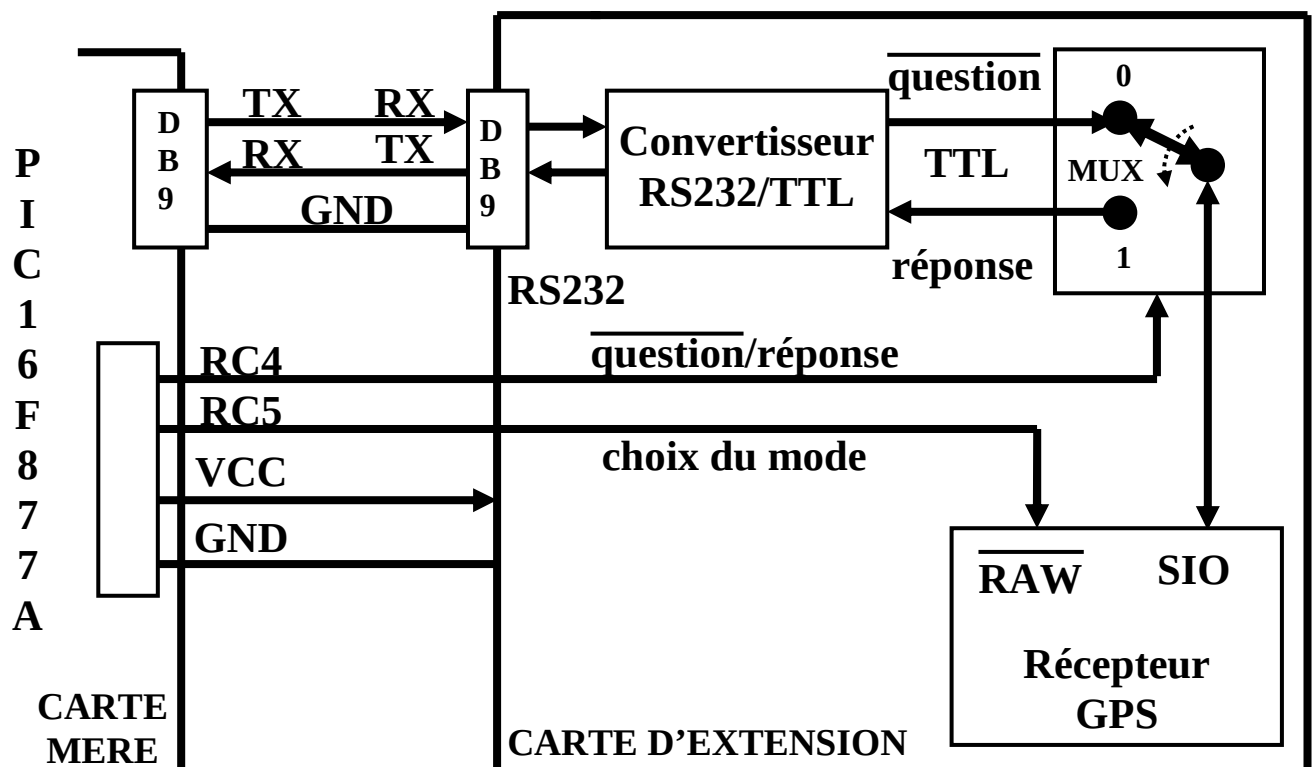


Figure 2.

La liaison série avec le **PIC** se fait sur **3 fils** au format **RS 232** (ou **V24**) :

- **TX** : Transmission des bits série du **PIC** vers le module **GPS**,
- **RX** : Réception par le **PIC** des bits série venant du module **GPS**,
- **GND** : la masse.

Les lignes **RC4** et **RC5** du **PORTC** du **PIC** sont aussi utilisées :

- **RC5** : permet le choix du mode de fonctionnement,
- **RC4** : (signal  $\overline{\text{question}}$ /réponse) commande le mutiplexeur à 2 positions **MUX** :
  - **RC4 = 0 : position 0**, pendant les phases de **question** (ou de commande),
  - **RC4 = 1 : position 1**, pendant les phases de **réponse** (à la commande).

Nous allons procéder périphérique par périphérique pour réaliser l'application complète de récupération des informations fournies par le récepteur **GPS**.

### 1. Etude la la liaison série.

### 1.1. Quelques éléments de base.

### 1.1.1. Comparer une liaison série par rapport à une liaison parallèle en terme avantage/inconvénient.

[illegible]

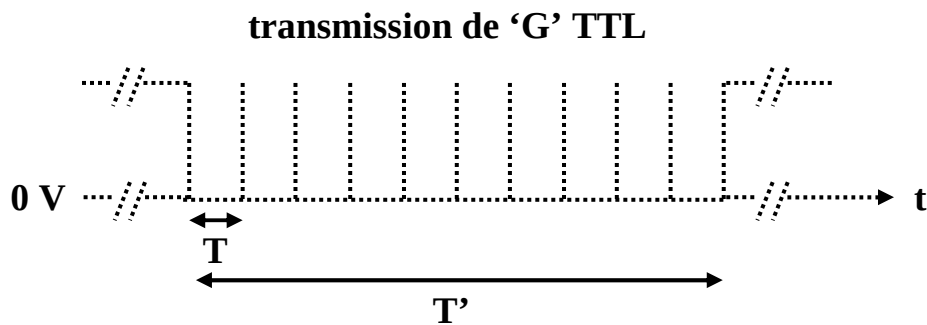
**1.1.2.** Quel est le principal intérêt de la norme **RS232** (ou **V24**) par rapport au standard **TTL** ?

[illegible]

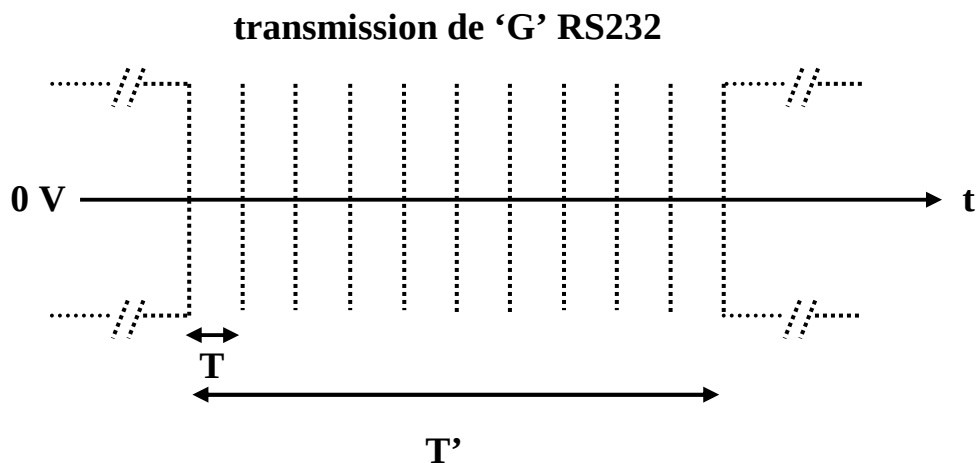
**1.1.3.** Compte tenu du format de la liaison série (et indiqué au début du texte de ce problème) nécessaire pour communiquer avec le module **GPS**, dessiner le chronogramme correspondant au transfert complet du caractère **ASCII 'G'**.

**a.** Valeur du caractère **ASCII 'G'** en binaire : \_\_\_\_\_

**b.** D'abord au standard **TTL** :



**c.** Puis au standard **RS232** :



**d.** Donner enfin la valeur de la durée  $T$  d'un bit, puis de  $T'$  correspondant à la durée complète prise par le transfert du caractère 'G' sur la ligne.

$T =$  \_\_\_\_\_

$T' =$  \_\_\_\_\_

## 1.2. Initialisation de la liaison série asynchrone côté PIC.

1.2.1. Indiquer les principaux registres accessibles à l'utilisateur et concernés par la liaison série asynchrone dans le PIC.

---

---

---

---

---

---

---

---

---

---

---

---

1.2.2. Calculer la valeur à écrire dans le registre **SPBRG** pour que la vitesse **V** de la liaison soit de **4800 bauds** en mode **high speed**.

On rappelle ci-dessous le tableau donné par **MICROCHIP** et on suppose que le **PIC** fonctionne à la fréquence **F<sub>OSC</sub> = 4 MHz**.

| BRGH = 0 (mode low speed)            | BRGH = 1 (mode high speed)           |
|--------------------------------------|--------------------------------------|
| $V = \frac{F_{osc}}{64.(SPBRG + 1)}$ | $V = \frac{F_{osc}}{16.(SPBRG + 1)}$ |

---

---

---

---

---

---

---

---

1.2.3. Indiquez la valeur des différents bits du registre **TXSTA** permettant de configurer la liaison série pour pouvoir émettre des caractères vers le module **GPS**.  
(mettre ci-dessous les justifications des valeurs des différents bits de **TXSTA**).

---

---

---

---

---

---

---

---

---

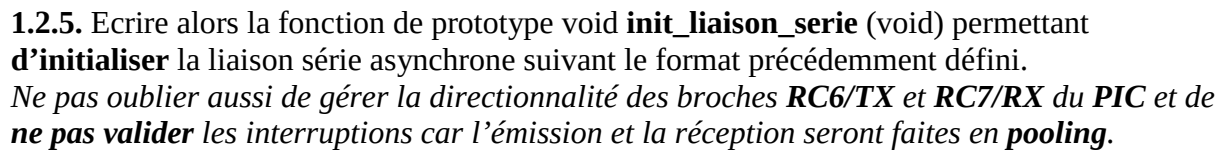
---

---

---

TXSTA

|      |     |      |      |   |      |      |      |
|------|-----|------|------|---|------|------|------|
|      |     |      |      |   |      |      |      |
| CSRC | TX9 | TXEN | SYNC | _ | BRGH | TRMT | TX9D |

[illegible]

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

### 1.3.Emission de caractères.

On décide d'utiliser la technique du pooling (ou scrutation) sur le bit **TXIF**.

**1.3.1.** Ecrire alors la fonction de prototype void **emet\_car** (unsigned char **car**) permettant d'émettre le caractère **car** sur la **liaison série asynchrone** du **PIC**.

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

**1.3.2.** En déduire la fonction de prototype void **emet\_string** (unsigned char \* message) permettant d'émettre le string **message** sur la **liaison série asynchrone** du **PIC**.

*On utilisera bien sûr la fonction **emet\_car** précédente (programmation structurée) mais on ne transmettra par le caractère final **0x00** de fin de string !!!*

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

### 1.4.Réception de caractères.

On décide d'utiliser la technique du pooling (ou scrutation) sur le bit **RCIF**.

Ecrire alors la fonction de prototype unsigned char **recoit\_car** (void) permettant de recevoir un caractère sur la **liaison série asynchrone** du **PIC** et de le retourner par cette fonction.

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

Les fonctions de base permettant d'utiliser la **liaison série asynchrone** du **PIC** étant maintenant écrites, nous allons passer à la gestion du mode de fonctionnement du récepteur **GPS**.

Mais même si vous n'avez pas su écrire le code de ces fonctions de base de la liaison série, vous pouvez toujours les utiliser par la suite, comme si elles étaient dans une bibliothèque !!!

## **2. Fonctionnement du récepteur GPS en mode SMART.**

Ecrire alors la fonction de prototype void **init\_gps\_mode\_smart** (void) permettant d'**initialiser** le récepteur **GPS** dans le mode **SMART**.

*Ne pas oublier de gérer la directionnalité des broches du **PORTC** utilisées.*

This image shows a full page of white paper with horizontal blue or grey ruling lines. The lines are evenly spaced and run across the width of the page, providing a template for handwriting practice or general writing. There are no margins, text, or other markings on the page.

### **3. Protocole question/réponse.**

On rappelle que dans le mode **SMART**, il faut tout d'abord envoyer une commande (suivant un certain format) vers le récepteur **GPS**.

En retour, ce dernier renvoie les informations demandées par cette commande.

Mais voyons cela plus en détail.

### Phase de requête ou de commande :

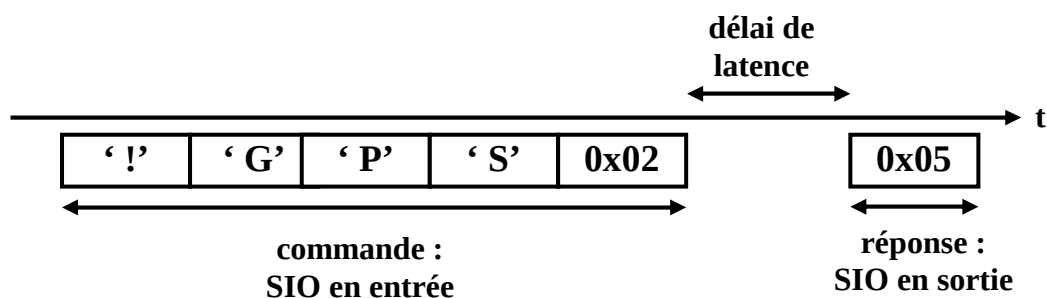
On envoie sur la broche **SIO** (qui fonctionne en **entrée**) les quatre caractères ASCII : **‘!’ ‘G’ ‘P’** et **‘S’** suivis d’un **numéro** identifiant le **type de commande** (cf. colonne ①).

| ①    |          |                                                                    | ④              |                                                                       |
|------|----------|--------------------------------------------------------------------|----------------|-----------------------------------------------------------------------|
| Cmd  | Constant | Description                                                        | Returned Bytes | Variables (1 Byte Each Unless Noted)                                  |
| 0x00 | GetInfo  | GPS Receiver Module version                                        | 2              | Hardware, Firmware                                                    |
| 0x01 | GetValid | Check validity of data string                                      | 1              | 0 = Not Valid, 1 = Valid                                              |
| 0x02 | GetSats  | Number of acquired satellites (12 maximum)                         | 1              |                                                                       |
| 0x03 | GetTime  | Time (UTC/Greenwich Mean Time)                                     | 3              | Hours, Minutes, Seconds                                               |
| 0x04 | GetDate  | Date (UTC/Greenwich Mean Time)                                     | 3              | Month, Day, Year                                                      |
| 0x05 | GetLat   | Latitude                                                           | 5              | Degrees, Minutes, Fractional Minutes (Word), Direction (0 = N, 1 = S) |
| 0x06 | GetLong  | Longitude                                                          | 5              | Degrees, Minutes, Fractional Minutes (Word), Direction (0 = E, 1 = W) |
| 0x07 | GetAlt   | Altitude above mean-sea-level (in tenths of meters, 65535 maximum) | 2              | Altitude (Word)                                                       |
| 0x08 | GetSpeed | Speed (in tenths of knots)                                         | 2              | Speed (Word)                                                          |
| 0x09 | GetHead  | Heading/direction of travel (in tenths of degrees)                 | 2              | Heading (Word)                                                        |

### Phase de réponse :

En retour, et après **un délai de latence non connu à priori**, le récepteur renvoie sur sa broche **SIO** (qui fonctionne alors en **sortie**) les informations demandées et au format précisé dans la colonne ④.

Voici par exemple, ce qu’on pourrait observer sur la broche **SIO** en utilisant la commande **0x02 (Getsats)** qui permet de savoir combien de satellites sont captés par le récepteur **GPS**. (dans cet exemple, on a supposé, de plus, que 5 satellites étaient vus par le récepteur **GPS**)



**3.1.** Définir 4 tableaux pour stocker l’**instant présent**, la **date**, la **latitude** et la **longitude** du lieu.

---

---

---

---

---

---

---

---

---

---

[illegible]

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

#### **4. Synthèse de l'application finale**

En utilisant les fonctions précédemment écrites, faire un main de synthèse permettant de savoir quel jour et quelle heure il est et où on se trouve, n'importe où dans le monde !!!

[illegible]

**ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!**

## **II. TESTS DU HARDWARE CARTE MULTIFONCTIONS**

Il est temps de consulter le schéma d'implantation de la carte multi fonctions situé dans votre répertoire de travail et dans :

[d\\_pic\\_ii/projets/documentations projets/docs carte multi fonction.](#)

Prenez connaissance de ce schéma d'implantation.

### **IMPORTANT**

**Avant de relier la carte GPS à la carte multi fonctions et de la mettre sous tension, voir avec l'enseignant pour valider le câblage.**

Pour avoir une idée sur ce qu'on demande de faire, sous **MPLAB** faire :

**File Import...** et aller chercher **dans votre répertoire** de travail dans le sous répertoire :

[d\\_pic\\_ii/projets/fichiers HEX OK projets](#)

le fichier : **gps mode smart.hex** (c'est un exécutable)

Puis, choisir éventuellement l'outil **MPLAB ICD3** (par **Debugger Select Tool**), envoyer cet exécutable sur l'ICD3 (par **Debugger Program**), et lancer ce programme...

**ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!**

### **III. PARTIE PRATIQUE GUIDEE**

Vous pouvez maintenant implanter les fonctions écrites dans la partie théorique guidée I précédente et les tester individuellement, quand c'est possible.

Puis, implantez le main (version de base) de la partie théorique et testez le.

Pensez aussi dans cette partie, à structurer votre projet en mettant ce qui est lié à votre périphérique :

- ce qui est lié au hardware et les prototypes de fonctions dans un fichier .h
- le code des fonctions dans un fichier .C

**ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!**

## **IV. PARTIE LIBRE EN PLUS GRANDE AUTONOMIE**

Utilisez le LCD pour afficher en temps réel les informations fournies par le GPS.

- date, heure,
- latitude, longitude
- altitude,
- nombre de satellites acquis...

en utilisant un appui sur le bouton poussoir RB0 pour passer d'une information à l'autre...

**ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!**

# **MODULATION MLI**

## I. PARTIE THEORIQUE GUIDEE

Avant de démarrer, prenez connaissance des pages 116 à 123 (pour le mode Output Compare du module CCP2 du PIC) du document :

Résumé du cours pendant TP PIC.pdf

situé dans le sous répertoire :

d\_pic\_ii/projets/Résumés cours

de votre répertoire de travail.

Le but de ce projet est de réaliser une modulation de type **MLI** (Modulation par Largeur d'Impulsions) sur un signal **analogique modulant sinusoïdal** pur  $V_m(t)$  d'amplitude  $E$  et de fréquence  $f$ .

On a donc :  $V_m(t) = E \cdot \cos(2\pi ft)$

On rappelle que, pour cette modulation, le signal **modulé MLI**, noté  $S(t)$  est :

- un signal **TTL**, de **période**  $T$  (et donc de **fréquence**  $F = \frac{1}{T} \gg f$ ) **constante**,
- et dont la **durée à l'état haut**  $TH_k$  sur chaque la  $k^{\text{ème}}$  période  $T$  dépend, à chaque instant  $t_k = kT$ , de la valeur du signal analogique  $V_m(t_k)$ . ( $k$  étant un entier naturel).

Ce qui est résumé sur la figure 1. ci-dessous, entre 2 instants quelconques :

$$t_k = kT \text{ et } t_{k+2} = (k+2)T$$

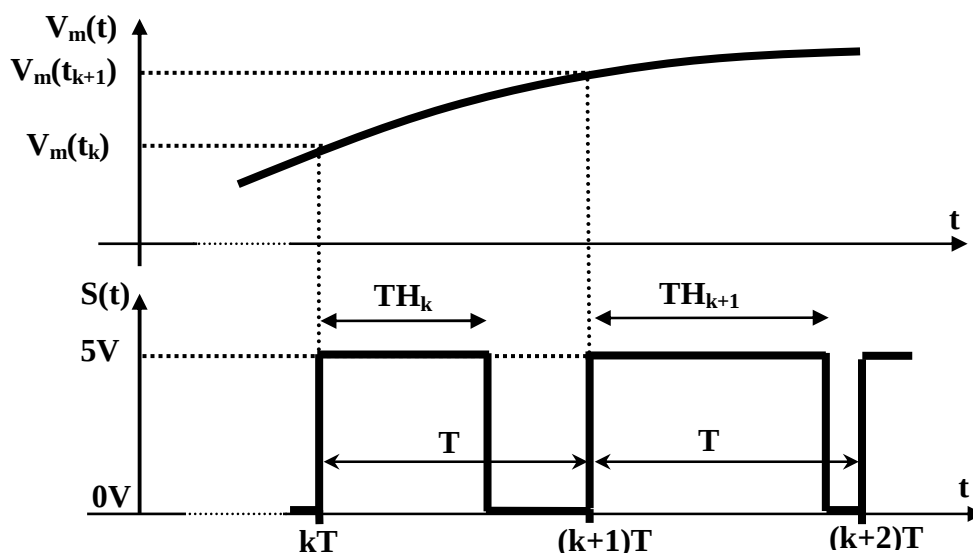


Figure 1.

Au final, le tracé d'oscilloscope de la figure 2. ci-dessous donne l'allure :

- du signal **modulant**  $V_m(t)$  : courbe du **haut**,
- du signal **modulé MLI**  $S(t)$  : courbe du **bas**.

On y voit que le **rapport cyclique**  $\alpha_k = \frac{TH_k}{T}$  n'est pas constant et **varie en permanence**, comme on l'a vu précédemment, au rythme du signal modulant  $V_m(t)$ .

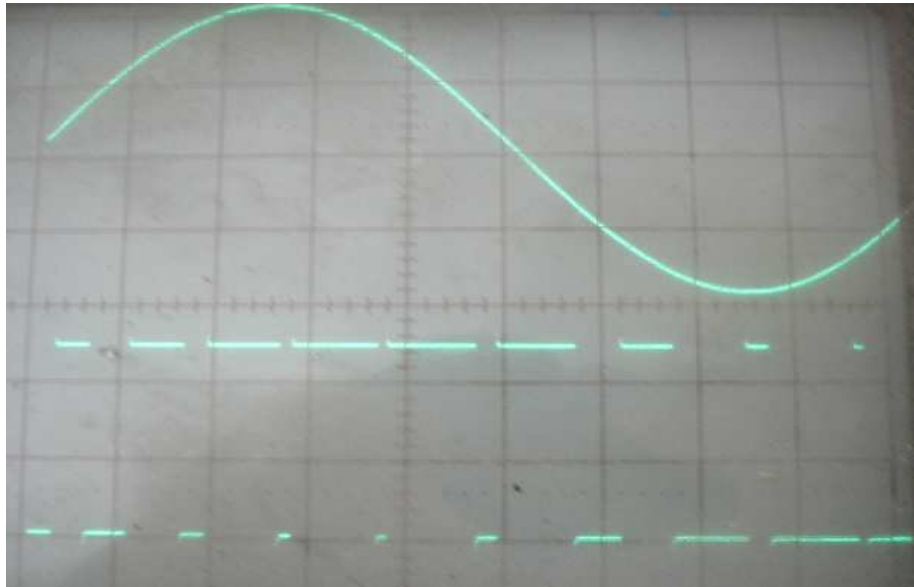


Figure 2.

Pour réaliser cette modulation, on va utiliser un **PIC 16F877** cadencé à **4 MHz** en mettant en œuvre :

- son **Convertisseur Analogique Numérique (CAN)** pour avoir une **image numérique** de la tension analogique  $V_m(t)$  à chaque instant **caractéristique**  $t_k = kT$ ,
- son module **CCP2**, utilisé dans son mode **OUTPUT COMPARE**, pour réaliser les différentes durées  $TH_k$  (**variables**) et la période  $T$  (**constante**) du signal modulé **MLI**  $S(t)$ .

Ces deux périphériques internes du PIC (**CAN** et **CCP2**) fonctionneront sous **interruption**.

Un synoptique simplifié du hardware est de plus fourni sur la figure 3. ci-dessous :

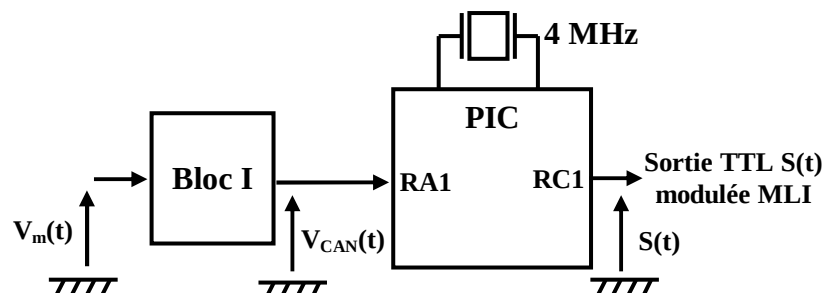


Figure 3.

## 1. Etude du Bloc I apparaissant sur la figure 3.

1.1. Compte tenues des **caractéristiques** de  $V_m(t)$ , expliquer pourquoi on ne peut pas **directement** injecter ce signal sur l'entrée RA1 du CAN.

---

---

---

---

Un peu d'électronique analogique.... : le schéma interne du **Bloc I** (d'entrée  $V_m(t)$  et de sortie  $V_{CAN}(t)$ ) est fourni sur la figure 4. suivante où les deux AOP sont supposés **idéaux** :

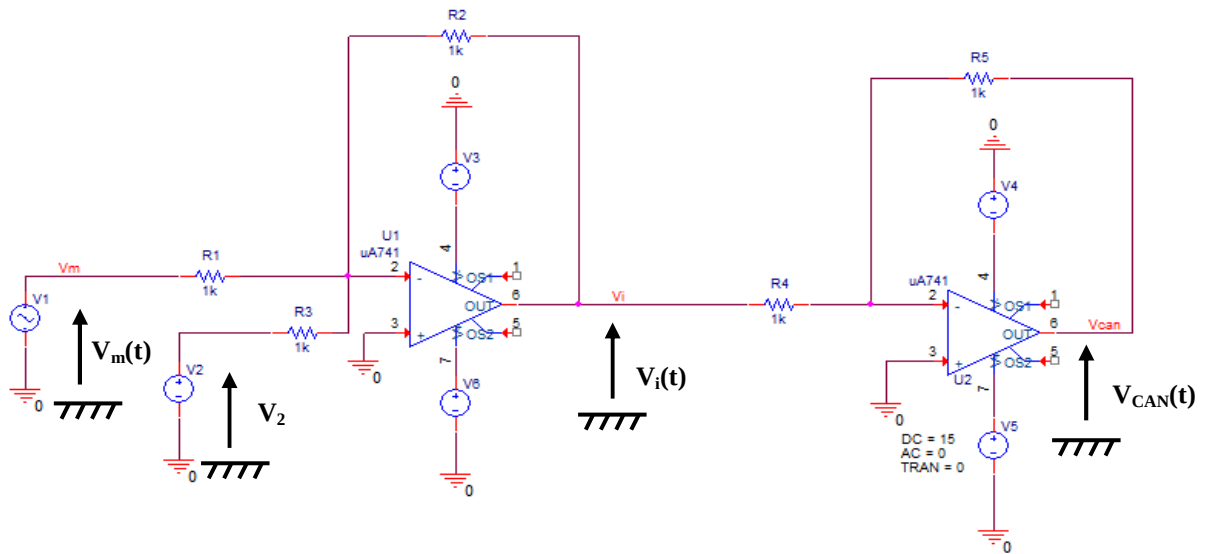


Figure 4.

1.2. Exprimer tout d'abord  $V_i(t)$  en fonction de  $V_m(t)$  et de la tension **continue**  $V_2$ .

---

---

---

---

---

1.3. Exprimer ensuite  $V_{CAN}(t)$  en fonction de  $V_i(t)$ .

---

---

---

---

1.4. En déduire enfin  $V_{CAN}(t)$  en fonction de  $V_m(t)$  et de  $V_2$ .

---

---

---

---

On s'impose  $E = 2 \text{ V}$  (amplitude de  $V_m(t)$ ,  $V_2 = 2,5 \text{ V}$  et  $f = 100 \text{ Hz}$ ).

On a réalisé une simulation de fonctionnement du **Bloc I** et obtenu les trois courbes (pour  $V_m(t)$ ,  $V_i(t)$  et  $V_{CAN}(t)$ ) représentées sur la figure 5. suivante :

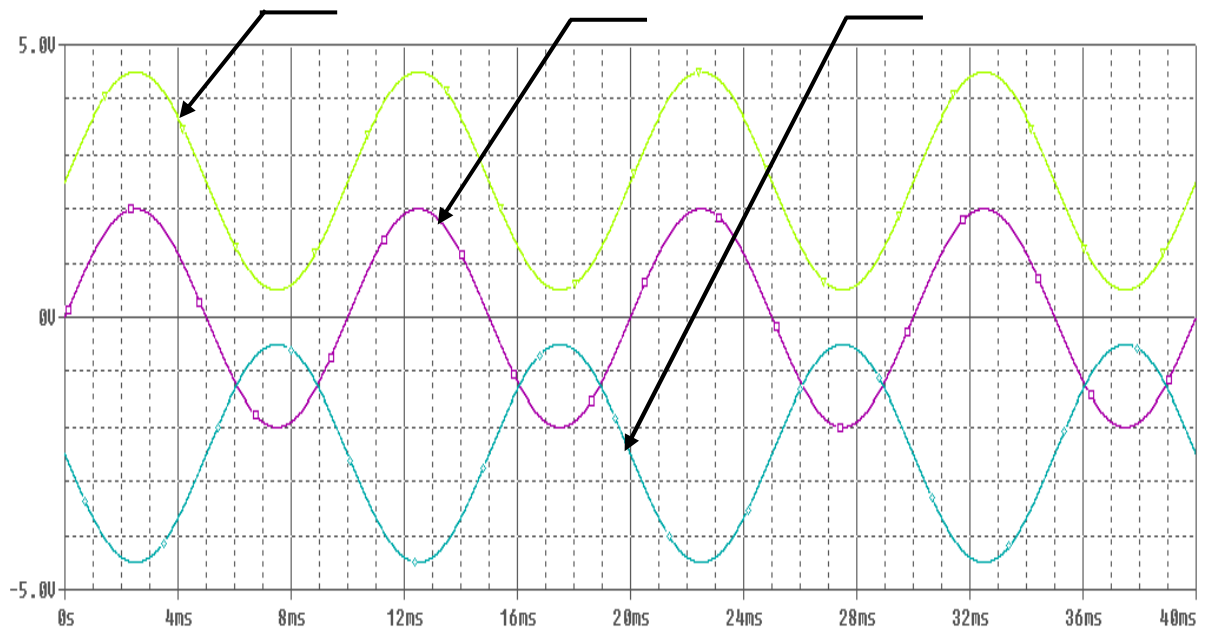


Figure 5.

**1.5.** En utilisant les résultats des questions 1.2. à 1.4. précédentes, compléter la figure 5. en repérant les 3 signaux  $V_m(t)$ ,  $V_i(t)$  et  $V_{CAN}(t)$  et montrer que le signal  $V_{CAN}(t)$  peut être relié sur l'entrée **RA1** du **CAN** du **PIC**, comme indiqué sur la figure 3.

---

---

---

---

---

---

---

---

---

---

---

---

**1.6. Conclusion :** donner un nom « **parlant et évocateur** » à ce **Bloc I**.

---

---

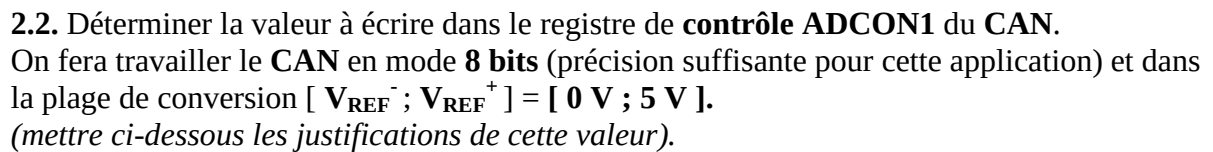
---

---

---

---

**2.1.** Déterminer la valeur à écrire dans le registre de **contrôle ADCON0** du **CAN**.  
(mettre ci-dessous les justifications de cette valeur).

[illegible][illegible]

---

---

---

---

---

---

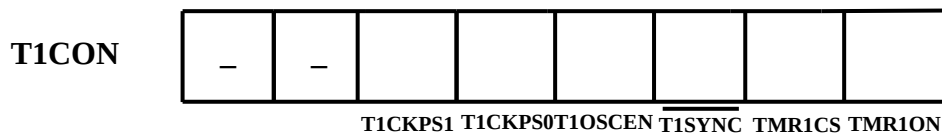
---

---

---

---

On rappelle que dans le mode **OUTPUT COMPARE**, le module **CCP2** du **PIC**, utilise le **TIMER1** comme **base de temps**.

[illegible]

**3.2.** Ecrire alors la fonction void **init\_timer1** (void) permettant **d'initialiser** le **TIMER1**.

---

---

---

---

---

---

---

---

On va utiliser le sous-mode **SPECIAL EVENT TRIGGER** du mode **OUTPUT COMPARE** du module **CCP2** (et paradoxalement **pas le mode PWM !!!**)

On rappelle que dans ce sous-mode, à **chaque comparaison valide** entre les registres 16 bits **CCPR2** et **TIMER1**, le **PIC automatiquement** :

- **met à 1** le flag **CCP2IF**,
- **remet à 0** le registre **TIMER1** (sans mettre à 1 le flag **TMR1IF** !),
- **lance une conversion** sur le **CAN** (s'il est en route),
- mais n'agit pas sur la broche **RC1** !!!

Ainsi, en travaillant sous interruption (pour le module **CCP2** et pour le **CAN**), on pourra gérer la durée **TH** à l'état haut du signal **S(t)**.

**3.3.** Indiquer la valeur des différents bits du registre **CCPCON2** permettant d'initialiser correctement le module **CCP2** en fonction des choix précédents.

*(mettre ci-dessous les justifications des valeurs des différents bits de **CCPCON2**).*

---

---

---

---

---

---

---

---



**3.4.** Ecrire alors la fonction void **init\_CCP2** (void) permettant **d'initialiser** le module **CCP2**. (ne pas oublier d'initialiser la broche **RC1** en sortie !!!)

---

---

---

---

---

---

---

---

#### 4. Gestion des masques d'interruption.

**4.1.** Indiquer les **différents masques d'interruption** à mettre en œuvre ici.

(**Rappel** : CCP2 et CAN fonctionnent sous interruption)

[illegible]

**4.2.** Ecrire la fonction void **init\_interruption** (void) permettant **d'initialiser** les masques d'interruption de la question précédente.

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and extend across the width of the page. There are no margins, text, or other markings on the paper.

### 5. La routine d'interruption.

On a vu que  $TH_k$  (durée à l'état haut de  $S(t)$  à l'instant  $t_k = kT$ ) dépendait de  $V_m(t_k)$ .

On souhaite réaliser une dépendance **linéaire**, représentée par la courbe de la figure 6.

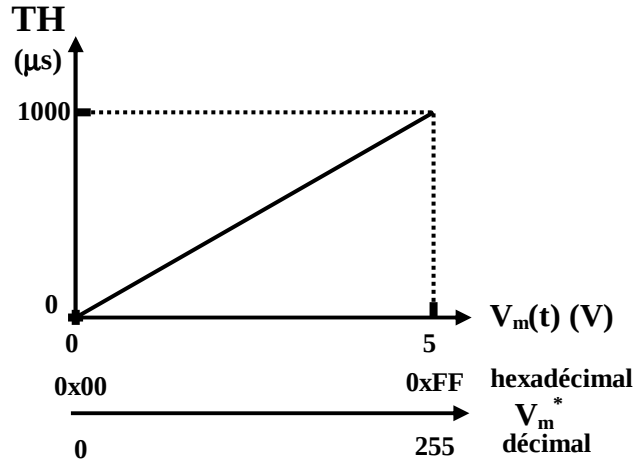


Figure 6.

On a également représenté sur cette courbe la valeur **numérique**  $V_m^*$  (correspondant aux différentes valeurs numériques de  $V_m(t_k)$ ) de la tension **analogique**  $V_m(t)$ .

5.1. Donner l'expression de  $TH_k$  en fonction de  $V_m(t_k)$ .

---

---

---

---

---

---

---

---

---

---

On choisit de prendre  $F = 1 \text{ kHz}$ , fréquence **constante** de  $S(t)$  et donc  $T = 1 \text{ ms} = 1000\mu\text{s}$ .

5.2. Ecrire la **routine d'interruption** correspondante.

(ne pas oublier qu'il faut gérer **deux causes** d'interruption : **CCP2** et **CAN** !!!)

---

---

---

---

---

---

---

---

---

---

[illegible]

## 6. Synthèse : modulation MLI.

En regroupant tout votre travail, proposer un applicatif permettant de réaliser la modulation **MLI** du signal  $V_m(t)$ .

This image shows a single sheet of white paper with horizontal blue or grey ruling lines. The lines are evenly spaced and run across the width of the page. There are no margins, text, or other markings on the paper.

## 7. Question subsidiaire...

Proposer une méthode (synoptique, algorithme...) pour réaliser la **démodulation** d'un signal modulé **MLI** comme **S(t)**.

[illegible]

**ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!**

## II. ESSAI DU HARDWARE

Il est temps de relier le GBF à la carte PIC. La fréquence **f** sera choisie à **100 Hz**.

Mais avant, il faut être sûr de ne pas envoyer une tension négative vers l'entrée **RA1** du CAN du PIC.

Il y a deux possibilités :

- la plus simple et la plus rapide : on règle correctement l'offset du GBF pour ne pas avoir à réaliser le bloc I de la partie théorique précédente : avec une vérification à l'oscilloscope en mode DC avant de relier le GBF à la carte PICDEM 2 PLUS,
- la plus longue : on câble le Bloc I de la partie théorique précédente sur une carte fille qui sera reliée à la carte PICDEM 2 PLUS : et dans ce cas là, on peut utiliser un signal sinusoïdal pur qu'on injectera sur cette carte fille.

### **IMPORTANT**

**Avant de relier le GBF, voir avec l'enseignant pour valider le câblage.**

Pour avoir une idée sur ce qu'on demande de faire, sous **MPLAB** faire :

**File**            **Import...**    et aller chercher **dans votre répertoire** de travail dans le sous répertoire :

**d\_pic\_ii/projets/fichiers HEX OK projets**

le fichier :    **mli\_avec\_output\_compare.hex**    (c'est un exécutable)

Puis, choisir éventuellement l'outil **MPLAB ICD3** (par **Debugger**        **Select Tool**), envoyer cet exécutable sur l'ICD3 (par **Debugger**    **Program**), et lancer ce programme...

Le signal modulé MLI sera visualisé sur la sortie **RC1** du PIC.

|                                                                    |
|--------------------------------------------------------------------|
| <b>ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!</b> |
|--------------------------------------------------------------------|

### **III. PARTIE PRATIQUE GUIDEE**

Vous pouvez maintenant implanter les fonctions écrites dans la partie théorique guidée I précédente et les tester individuellement, quand c'est possible.

Puis, implantez le main (version de base) de la partie théorique et testez le.

Pensez aussi dans cette partie, à structurer votre projet en mettant ce qui est lié à votre périphérique :

- ce qui est lié au hardware et les prototypes de fonctions dans un fichier .h
- le code des fonctions dans un fichier .C

**ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!**

#### **IV. PARTIE LIBRE EN PLUS GRANDE AUTONOMIE**

Dans cette partie, vous pouvez réaliser une carte fille pour réaliser une démodulation du signal MLI.

L'idée pourrait être d'utiliser la fonction **INPUT CAPTURE** du PIC puis un Convertisseur Numérique Analogique pour retrouver le signal modulant d'entrée.

On pourra aussi afficher la fréquence  $f$  sur le LCD.

**ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!**

# **GRADATEUR DE LUMIERE**

## I. PARTIE THEORIQUE GUIDEE

Avant de démarrer, prenez connaissance des pages 124 à 131 (pour le mode PWM du PIC) du document :

Résumé du cours pendant TP PIC.pdf

situé dans le sous répertoire :

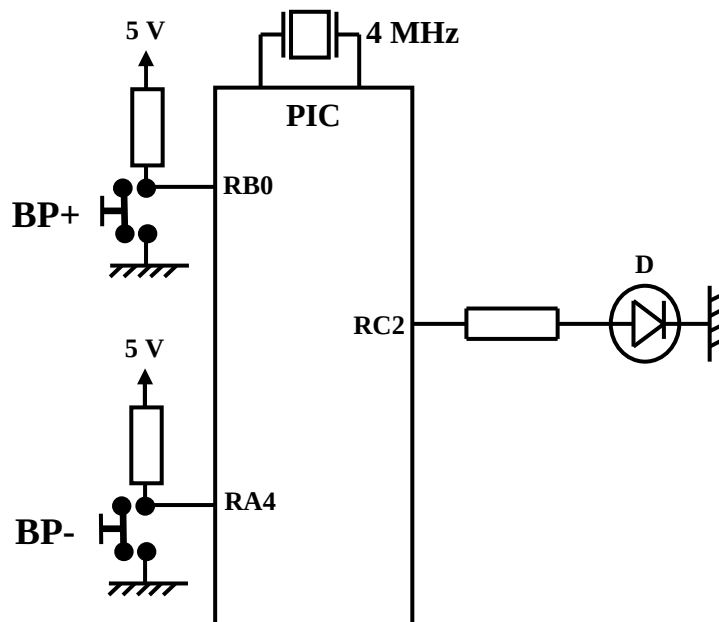
d\_pic\_ii/projets/Résumés cours

de votre répertoire de travail.

On souhaite réaliser dans ce projet un petit **gradateur de lumière** commandable par boutons poussoirs. Le schéma électronique simplifié de ce système est fourni sur la figure ci-dessous.

Les deux boutons poussoirs **BP+** et **BP-** sont à contact fugitif. Ils mettent :

- soit au **0 V** les broches **RB0** et **RA4** lorsqu'on **appuie** dessus,
- soit au **5 V** les broches **RB0** et **RA4** quand on les **relâche**.



Le fonctionnement souhaité est assez simple :

A la mise sous tension la led **D** est **éteinte**.

L'appui sur le bouton poussoir **BP+** l'allume **progressivement** et **de plus en plus** en fonction de la **durée** de cet appui, jusqu'à son **maximum d'éclairement**.

Et inversement, l'appui sur le bouton poussoir **BP-** l'éteint **progressivement** et **de plus en plus** en fonction de la **durée** de cet appui, jusqu'à son **extinction totale**.

Ainsi, en jouant sur l'un ou l'autre de ces deux boutons poussoirs on peut régler la luminosité à la valeur souhaitée.

Pour obtenir ce résultat, on utilisera le mode de fonctionnement en **PWM** du module **CCP1**.

## 1. Gestion des lignes des PORTS utilisés.

Ecrire la fonction de prototype void **init\_PORT** (void) permettant de choisir la **directionnalité** :

- de la broche **RA4** (du **PORTA**),
- de la broche **RB0** (du **PORTB**),
- et de la broche **RC2** (du **PORTC**).

et elles seules !!!

---

---

---

---

---

---

---

---

---

---

## 2. Gestion du TIMER2 du PIC.

On souhaite prendre **T = 1 ms** (**période** du signal **PWM**).

**2.1.** Calculer la valeur à écrire dans le registre **PR2** du **TIMER2** sachant que le quartz du **PIC** est à **4 MHz**.

---

---

---

---

---

---

---

---

**2.2.** En déduire la valeur à écrire dans le registre **T2CON** pour initialiser le **TIMER2**.

---

---

---

---

---

---

---

---

**2.3.** Ecrire alors la fonction de prototype void **init\_timer2** (void) permettant d'initialiser le **TIMER2** en fonction des résultats obtenus dans les deux questions précédentes.

---

---

---

---

---

---

---

---

### **3. Mode PWM.**

**3.1.** Rappeler en quelques lignes ou par un schéma le principe d'une commande **PWM** en mettant en évidence sa période **T** et sa durée  $\tau$  à l'état haut.

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

**3.2.** Donner la valeur à écrire dans le registre **CCP1CON** du module **CCP1** pour obtenir le fonctionnement en **PWM**.

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

**3.3.** En déduire la fonction de prototype void **init\_ccp1** (void) permettant d'initialiser le module **CCP1** en se rappelant également qu'à la mise sous tension la led **D** est éteinte.

---

---

---

---

---

---

---

---

---

---

#### **4. Gestion de la durée des appuis sur les boutons poussoirs.**

La gestion des deux boutons poussoirs **SP+** et **SP-** sera faite par pooling (scrutation).

Or, la durée d'un appui même très bref (quelques centaines de ms) sur un bouton poussoir étant toujours très grande par rapport à celle de la boucle de scrutation de ce bouton poussoir, il est nécessaire de rajouter une temporisation après la détection de cet appui.

On utilisera le **TIMER0** du **PIC** pour réaliser cette temporisation paramétrable.

**4.1.** Déterminer la **Valeur Initiale VI** à écrire dans le registre **compteur TMR0** du **TIMER0** pour obtenir une temporisation de **1 ms**. (on prendra la valeur la plus petite pour le **prédiviseur p**).

---

---

---

---

---

---

---

---

---

---

**4.2.** En déduire la valeur à écrire dans le registre de **contrôle OPTION\_REG** (reconnu sous le nom **OPTION** par le compilateur C) du **TIMER0** pour obtenir cette temporisation.

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

**4.3.** Ecrire alors la fonction de prototype void **init\_timer0** (void) permettant **d'initialiser** le **TIMER0**.

---

---

---

---

---

---

---

**4.4.** Ecrire ensuite la fonction de prototype void **tempo\_1\_ms** (void) (venant scruter le bit **T0IF**) et dont la durée d'exécution **fixe** est de **1 ms**.

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

**4.5.** En utilisant cette fonction **tempo\_1\_ms**, écrire la fonction **paramétrable** de prototype : void **tempo\_p\_ms** (int **duree\_en ms** ) pour réaliser une temporisation variable de **duree\_en\_ms** millisecondes.

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

---

### 5. Ecriture de l'applicatif.

Proposez enfin un programme pour réaliser le petit gradateur de lumière.

[illegible]

**ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!**

## II. TESTS DU HARDWARE

Il est temps de rajouter une led et une résistance sur la carte PICDEM 2 PLUS branchées sur la sortie **RC2** du PIC.

### **IMPORTANT**

**Avant de mettre la carte PICDEM 2 PLUS sous tension, voir avec l'enseignant pour valider le câblage.**

Pour avoir une idée sur ce qu'on demande de faire, sous **MPLAB** faire :

**File**            **Import...**        et aller chercher **dans votre répertoire** de travail dans le sous répertoire :

**d\_pic\_ii/projets/fichiers HEX OK projets**

le fichier :    **gradateur.hex**            (c'est un exécutable)

Puis, choisir éventuellement l'outil **MPLAB ICD3** (par **Debugger**        **Select Tool**), envoyer cet exécutable sur l'ICD3 (par **Debugger** **Program**), et lancer ce programme...

|                                                                    |
|--------------------------------------------------------------------|
| <b>ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!</b> |
|--------------------------------------------------------------------|

### **III. PARTIE PRATIQUE GUIDEE**

Vous pouvez maintenant implanter les fonctions écrites dans la partie théorique guidée I précédente et les tester individuellement, quand c'est possible.

Puis, implantez le main (version de base) de la partie théorique et testez le.

Pensez aussi dans cette partie, à structurer votre projet en mettant ce qui est lié à votre périphérique :

- ce qui est lié au hardware et les prototypes de fonctions dans un fichier .h
- le code des fonctions dans un fichier .C

**ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!**

#### **IV. PARTIE LIBRE EN PLUS GRANDE AUTONOMIE**

Utilisez le LCD pour afficher en temps réel la valeur du rapport cyclique en %.

**ON APPELERA L'ENSEIGNANT POUR LUI MONTRER SES RESULTATS !!!**